

RNA-seq analysis in R

(index.html)

Differential expression analysis

Authors: Belinda Phipson, Anna Trigos, Matt Ritchie, Maria Doyle, Harriet Dashnow, Charity Law

Resources and data files

This material has been created using the following resources:

<http://www.statsci.org/smyth/pubs/QLedgeRPreprint.pdf> (<http://www.statsci.org/smyth/pubs/QLedgeRPreprint.pdf>) (Lun, Chen, and Smyth 2016)

http://monashbioinformaticsplatform.github.io/RNAseq-DE-analysis-with-R/99-RNAseq_DE_analysis_with_R.html

(http://monashbioinformaticsplatform.github.io/RNAseq-DE-analysis-with-R/99-RNAseq_DE_analysis_with_R.html)

Data files downloaded from:

ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE60nnn/GSE60450/suppl/GSE60450_Lactation-GenewiseCounts.txt.gz

(ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE60nnn/GSE60450/suppl/GSE60450_Lactation-GenewiseCounts.txt.gz)

http://bioinf.wehi.edu.au/software/MSigDB/mouse_c2_v5.rdata (http://bioinf.wehi.edu.au/software/MSigDB/mouse_c2_v5.rdata)

http://bioinf.wehi.edu.au/software/MSigDB/mouse_H_v5.rdata (http://bioinf.wehi.edu.au/software/MSigDB/mouse_H_v5.rdata)

Data files are available from: <https://figshare.com/s/1d788fd384d33e913a2a> (<https://figshare.com/s/1d788fd384d33e913a2a>) You should download these files and place them in your `/data` directory.

Data files:

sampleinfo.txt SampleInfo_Corrected.txt GSE60450_Lactation-GenewiseCounts.txt

mouse_c2_v5.rdata

mouse_H_v5.rdata ResultsTable_small.txt small_counts.txt

Packages used: limma, edgeR, gplots, org.Mm.eg.db, RColorBrewer, Glimma

Overview

- Reading in table of counts
- Filtering lowly expressed genes
- Quality control
- Normalisation for composition bias
- Differential expression analysis
- Testing relative to a threshold
- Visualisation
- Gene set testing

Introduction and data import

Measuring gene expression on a genome-wide scale has become common practice over the last two decades or so, with microarrays predominantly used pre-2008. With the advent of next generation sequencing technology in 2008, an increasing number of scientists use this technology to measure and understand changes in gene expression in often complex systems. As sequencing costs have decreased, using RNA-Seq to simultaneously measure the expression of tens of thousands of genes for multiple samples has never been easier. The cost of these experiments has now moved from generating the data to storing and analysing it.

There are many steps involved in analysing an RNA-Seq experiment. Analysing an RNAseq experiment begins with sequencing reads. These are aligned to a reference genome, then the number of reads mapped to each gene can be counted. This results in a table of counts, which is what we perform statistical analyses on in R. While mapping and counting are important and necessary tasks, today we will be starting from the count data and getting stuck into analysis.

First, let's load all the packages we will need to analyse the data.

```
library(edgeR)
library(limma)
library(Glimma)
library(gplots)
library(org.Mm.eg.db)
library(RColorBrewer)
```

Mouse mammary gland dataset

The data for this tutorial comes from a Nature Cell Biology paper, *EGF-mediated induction of Mcl-1 at the switch to lactation is essential for alveolar cell survival* (<http://www.ncbi.nlm.nih.gov/pubmed/25730472>) (Fu et al. 2015). Both the raw data (sequence reads) and processed data (counts) can be downloaded from Gene Expression Omnibus database (GEO) under accession number GSE60450 (<http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE60450>).

This study examines the expression profiles of basal stem-cell enriched cells (B) and committed luminal cells (L) in the mammary gland of virgin, pregnant and lactating mice. Six groups are present, with one for each combination of cell type and mouse status. Each group contains two biological replicates. We will first use the counts file as a starting point for our analysis. This data has already been aligned to the mouse genome. The command line tool featureCounts (Liao, Smyth, and Shi 2014) was used to count reads mapped to mouse genes from Refseq annotation (see the paper (<http://www.ncbi.nlm.nih.gov/pubmed/25730472>) for details).

Reading in the data

Set up an RStudio project specifying the directory where you have saved the `/data` directory. Download and read in the data.

```
# Read the data into R
seqdata <- read.delim("data/GSE60450_Lactation-GenewiseCounts.txt", stringsAsFactors = FALSE)
# Read the sample information into R
sampleinfo <- read.delim("data/SampleInfo.txt")
```

Let's take a look at the data. You can use the `head` command to see the first 6 lines. The `dim` command will tell you how many rows and columns the data frame has.

```
head(seqdata)
```

EntrezGeneID	Length	MCL1.DG_BC2TUACXX_ACTTGA_L002_R1	
1	497097	3634	438
2	100503874	3259	1
3	100038431	1634	0
4	19888	9747	1
5	20671	3130	106
6	27395	4203	309
MCL1.DH_BC2TUACXX_CAGATC_L002_R1 MCL1.DI_BC2TUACXX_ACAGTG_L002_R1			
1		300	65
2		0	1
3		0	0
4		1	0
5		182	82
6		234	337
MCL1.DJ_BC2TUACXX_CGATGT_L002_R1 MCL1.DK_BC2TUACXX_TTAGGC_L002_R1			
1		237	354
2		1	0
3		0	0
4		0	0
5		105	43
6		300	290
MCL1.DL_BC2TUACXX_ATCACG_L002_R1 MCL1.LA_BC2TUACXX_GATCAG_L001_R1			
1		287	0
2		4	0
3		0	0
4		0	10
5		82	16
6		270	560
MCL1.LB_BC2TUACXX_TGACCA_L001_R1 MCL1.LC_BC2TUACXX_GCCAAT_L001_R1			
1		0	0
2		0	0
3		0	0
4		3	10
5		25	18
6		464	489
MCL1.LD_BC2TUACXX_GGCTAC_L001_R1 MCL1.LE_BC2TUACXX_TAGCTT_L001_R1			
1		0	0
2		0	0
3		0	0
4		2	0
5		8	3
6		328	307
MCL1.LF_BC2TUACXX_CTTGTA_L001_R1			
1		0	
2		0	
3		0	
4		0	
5		10	
6		342	

```
dim(seqdata)
```

```
[1] 27179    14
```

The `seqdata` object contains information about genes (one gene per row), the first column has the Entrez gene id, the second has the gene length and the remaining columns contain information about the number of reads aligning to the gene in each experimental sample. There are two replicates for each cell type and timepoint (detailed sample info can be found in file "GSE60450_series_matrix.txt" from the GEO website (<http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE60450>)). The sampleinfo file contains basic information about the samples that we will need for the analysis today.

```
sampleinfo
```

	FileName	SampleName	CellType	Status
1	MCL1.DG_BC2CTUACXX_ACTTGA_L002_R1		MCL1.DG	luminal virgin
2	MCL1.DH_BC2CTUACXX_CAGATC_L002_R1		MCL1.DH	basal virgin
3	MCL1.DI_BC2CTUACXX_ACAGTG_L002_R1		MCL1.DI	basal pregnant
4	MCL1.DJ_BC2CTUACXX_CGATGT_L002_R1		MCL1.DJ	basal pregnant
5	MCL1.DK_BC2CTUACXX_TTAGGC_L002_R1		MCL1.DK	basal lactate
6	MCL1.DL_BC2CTUACXX_ATCACG_L002_R1		MCL1.DL	basal lactate
7	MCL1.LA_BC2CTUACXX_GATCAG_L001_R1		MCL1.LA	basal virgin
8	MCL1.LB_BC2CTUACXX_TGACCA_L001_R1		MCL1.LB	luminal virgin
9	MCL1.LC_BC2CTUACXX_GCCAAT_L001_R1		MCL1.LC	luminal pregnant
10	MCL1.LD_BC2CTUACXX_GGCTAC_L001_R1		MCL1.LD	luminal pregnant
11	MCL1.LE_BC2CTUACXX_TAGCTT_L001_R1		MCL1.LE	luminal lactate
12	MCL1.LF_BC2CTUACXX_CTTGTA_L001_R1		MCL1.LF	luminal lactate

We will be manipulating and reformatting the counts matrix into a suitable format for downstream analysis. The first two columns in the `seqdata` dataframe contain annotation information. We need to make a new matrix containing only the counts, but we can store the gene identifiers (the `EntrezGeneID` column) as rownames. We will add more annotation information about each gene later on in the workshop.

Format the data

Let's create a new data object, `countdata`, that contains only the counts for the 12 samples.

```
# Remove first two columns from seqdata
countdata <- seqdata[, -(1:2)]
# Look at the output
head(countdata)
```

```

MCL1.DG_BC2TUACXX_ACTTGA_L002_R1 MCL1.DH_BC2TUACXX_CAGATC_L002_R1
1 438 300
2 1 0
3 0 0
4 1 1
5 106 182
6 309 234
MCL1.DI_BC2TUACXX_ACAGTG_L002_R1 MCL1.DJ_BC2TUACXX_CGATGT_L002_R1
1 65 237
2 1 1
3 0 0
4 0 0
5 82 105
6 337 300
MCL1.DK_BC2TUACXX_TTAGGC_L002_R1 MCL1.DL_BC2TUACXX_ATCACG_L002_R1
1 354 287
2 0 4
3 0 0
4 0 0
5 43 82
6 290 270
MCL1.LA_BC2TUACXX_GATCAG_L001_R1 MCL1.LB_BC2TUACXX_TGACCA_L001_R1
1 0 0
2 0 0
3 0 0
4 10 3
5 16 25
6 560 464
MCL1.LC_BC2TUACXX_GCCAAT_L001_R1 MCL1.LD_BC2TUACXX_GGCTAC_L001_R1
1 0 0
2 0 0
3 0 0
4 10 2
5 18 8
6 489 328
MCL1.LE_BC2TUACXX_TAGCTT_L001_R1 MCL1.LF_BC2TUACXX_CTTGTA_L001_R1
1 0 0
2 0 0
3 0 0
4 0 0
5 3 10
6 307 342

```

```

# Store EntrezGeneID as rownames
rownames(countdata) <- seqdata[,1]

```

Take a look at the output

```
head(countdata)
```

```

MCL1.DG_BC2CTUACXX_ACTTGA_L002_R1
497097 438
100503874 1
100038431 0
19888 1
20671 106
27395 309
MCL1.DH_BC2CTUACXX_CAGATC_L002_R1
497097 300
100503874 0
100038431 0
19888 1
20671 182
27395 234
MCL1.DI_BC2CTUACXX_ACAGTG_L002_R1
497097 65
100503874 1
100038431 0
19888 0
20671 82
27395 337
MCL1.DJ_BC2CTUACXX_CGATGT_L002_R1
497097 237
100503874 1
100038431 0
19888 0
20671 105
27395 300
MCL1.DK_BC2CTUACXX_TTAGGC_L002_R1
497097 354
100503874 0
100038431 0
19888 0
20671 43
27395 290
MCL1.DL_BC2CTUACXX_ATCACG_L002_R1
497097 287
100503874 4
100038431 0
19888 0
20671 82
27395 270
MCL1.LA_BC2CTUACXX_GATCAG_L001_R1
497097 0
100503874 0
100038431 0
19888 10
20671 16
27395 560
MCL1.LB_BC2CTUACXX_TGACCA_L001_R1
497097 0
100503874 0
100038431 0
19888 3
20671 25
27395 464
MCL1.LC_BC2CTUACXX_GCCAAT_L001_R1
497097 0
100503874 0
100038431 0
19888 10
20671 18
27395 489
MCL1.LD_BC2CTUACXX_GGCTAC_L001_R1
497097 0
100503874 0
100038431 0
19888 2
20671 8
27395 328
MCL1.LE_BC2CTUACXX_TAGCTT_L001_R1

```

```

497097      0
100503874   0
100038431   0
19888       0
20671       3
27395      307
      MCL1.LF_BC2TUACXX_CTTGTA_L001_R1
497097      0
100503874   0
100038431   0
19888       0
20671      10
27395     342

```

Now take a look at the column names

```
colnames(countdata)
```

```

[1] "MCL1.DG_BC2TUACXX_ACTTGA_L002_R1"
[2] "MCL1.DH_BC2TUACXX_CAGATC_L002_R1"
[3] "MCL1.DI_BC2TUACXX_ACAGTG_L002_R1"
[4] "MCL1.DJ_BC2TUACXX_CGATGT_L002_R1"
[5] "MCL1.DK_BC2TUACXX_TTAGGC_L002_R1"
[6] "MCL1.DL_BC2TUACXX_ATCACG_L002_R1"
[7] "MCL1.LA_BC2TUACXX_GATCAG_L001_R1"
[8] "MCL1.LB_BC2TUACXX_TGACCA_L001_R1"
[9] "MCL1.LC_BC2TUACXX_GCCAAT_L001_R1"
[10] "MCL1.LD_BC2TUACXX_GGCTAC_L001_R1"
[11] "MCL1.LE_BC2TUACXX_TAGCTT_L001_R1"
[12] "MCL1.LF_BC2TUACXX_CTTGTA_L001_R1"

```

These are the sample names which are pretty long so we'll shorten these to contain only the relevant information about each sample. We will use the `substr` command to extract the first 7 characters and use these as the colnames.

```

# using substr, you extract the characters starting at position 1 and stopping at position 7 of the colnames
colnames(countdata) <- substr(colnames(countdata),start=1,stop=7)

```

Take a look at the output

```
head(countdata)
```

```

MCL1.DG MCL1.DH MCL1.DI MCL1.DJ MCL1.DK MCL1.DL MCL1.LA MCL1.LB
497097      438      300      65      237      354      287      0      0
100503874    1       0       1       1       0       4       0      0
100038431    0       0       0       0       0       0       0      0
19888        1       1       0       0       0       0     10      3
20671       106     182     82     105     43     82     16     25
27395       309     234     337     300     290     270     560    464

      MCL1.LC MCL1.LD MCL1.LE MCL1.LF
497097      0       0       0       0
100503874    0       0       0       0
100038431    0       0       0       0
19888       10       2       0       0
20671       18       8       3     10
27395      489     328     307     342

```

Note that the column names are now the same as `SampleName` in the `sampleinfo` file. This is good because it means our sample information in `sampleinfo` is in the same order as the columns in `countdata`.

```
table(colnames(countdata)==sampleinfo$SampleName)
```

```

TRUE
12

```

Filtering to remove lowly expressed genes

Genes with very low counts across all libraries provide little evidence for differential expression and they interfere with some of the statistical approximations that are used later in the pipeline. They also add to the multiple testing burden when estimating false discovery rates, reducing power to detect differentially expressed genes. These genes should be filtered out prior to further analysis.

There are a few ways to filter out lowly expressed genes. When there are biological replicates in each group, in this case we have a sample size of 2 in each group, we favour filtering on a minimum counts per million threshold present in at least 2 samples. Two represents the smallest sample size for each group in our experiment. In this dataset, we choose to retain genes if they are expressed at a counts-per-million (CPM) above 0.5 in at least two samples.

We'll use the `cpm` function from the *edgeR* library (M D Robinson, McCarthy, and Smyth 2010) to generate the CPM values and then filter. Note that by converting to CPMs we are normalising for the different sequencing depths for each sample.

```
# Obtain CPMs
myCPM <- cpm(countdata)
# Have a look at the output
head(myCPM)
```

```
      MCL1.DG  MCL1.DH  MCL1.DI  MCL1.DJ  MCL1.DK
497097  18.85684388 13.77543859 2.69700983 10.45648006 16.442685
100503874 0.04305215 0.00000000 0.04149246 0.04412017 0.000000
100038431 0.00000000 0.00000000 0.00000000 0.00000000 0.000000
19888    0.04305215 0.04591813 0.00000000 0.00000000 0.000000
20671    4.56352843 8.35709941 3.40238163 4.63261775 1.997275
27395    13.30311589 10.74484210 13.98295863 13.23605071 13.469996
      MCL1.DL  MCL1.LA  MCL1.LB  MCL1.LC  MCL1.LD
497097  14.3389690 0.0000000 0.0000000 0.0000000 0.0000000
100503874 0.1998463 0.0000000 0.0000000 0.0000000 0.0000000
100038431 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
19888    0.0000000 0.4903857 0.1381969 0.4496078 0.09095771
20671    4.0968483 0.7846171 1.1516411 0.8092940 0.36383085
27395    13.4896224 27.4615975 21.3744588 21.9858214 14.91706476
      MCL1.LE  MCL1.LF
497097  0.0000000 0.0000000
100503874 0.0000000 0.0000000
100038431 0.0000000 0.0000000
19888    0.0000000 0.0000000
20671    0.1213404 0.4055595
27395    12.4171715 13.8701357
```

```
# Which values in myCPM are greater than 0.5?
thresh <- myCPM > 0.5
# This produces a logical matrix with TRUEs and FALSEs
head(thresh)
```

```
      MCL1.DG MCL1.DH MCL1.DI MCL1.DJ MCL1.DK MCL1.DL MCL1.LA MCL1.LB
497097      TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  FALSE  FALSE
100503874 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
100038431 FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
19888      FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
20671      TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
27395      TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
      MCL1.LC MCL1.LD MCL1.LE MCL1.LF
497097 FALSE FALSE FALSE FALSE
100503874 FALSE FALSE FALSE FALSE
100038431 FALSE FALSE FALSE FALSE
19888      FALSE FALSE FALSE FALSE
20671      TRUE FALSE FALSE FALSE
27395      TRUE  TRUE  TRUE  TRUE
```

```
# Summary of how many TRUEs there are in each row
# There are 11433 genes that have TRUEs in all 12 samples.
table(rowSums(thresh))
```



```

      0      1      2      3      4      5      6      7      8      9     10     11
10857  518   544   307   346   307   652   323   547   343   579   423
12
11433

```

```

# we would like to keep genes that have at least 2 TRUES in each row of thresh
keep <- rowSums(thresh) >= 2
# Subset the rows of countdata to keep the more highly expressed genes
counts.keep <- countdata[keep,]
summary(keep)

```

```

Mode   FALSE   TRUE
logical 11375  15804

```

```
dim(counts.keep)
```

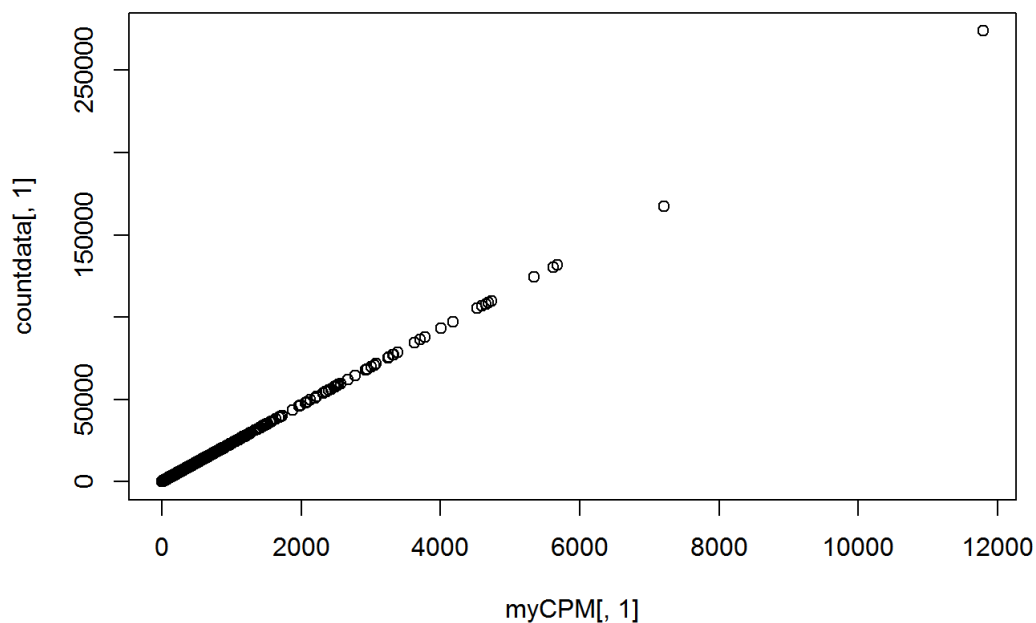
```
[1] 15804   12
```

A CPM of 0.5 is used as it corresponds to a count of 10-15 for the library sizes in this data set. If the count is any smaller, it is considered to be very low, indicating that the associated gene is not expressed in that sample. A requirement for expression in two or more libraries is used as each group contains two replicates. This ensures that a gene will be retained if it is only expressed in one group. Smaller CPM thresholds are usually appropriate for larger libraries. As a general rule, a good threshold can be chosen by identifying the CPM that corresponds to a count of 10, which in this case is about 0.5. You should filter with CPMs rather than filtering on the counts directly, as the latter does not account for differences in library sizes between samples.

```

# Let's have a look and see whether our threshold of 0.5 does indeed correspond to a count of about 10-15
# We will look at the first sample
plot(myCPM[,1],countdata[,1])

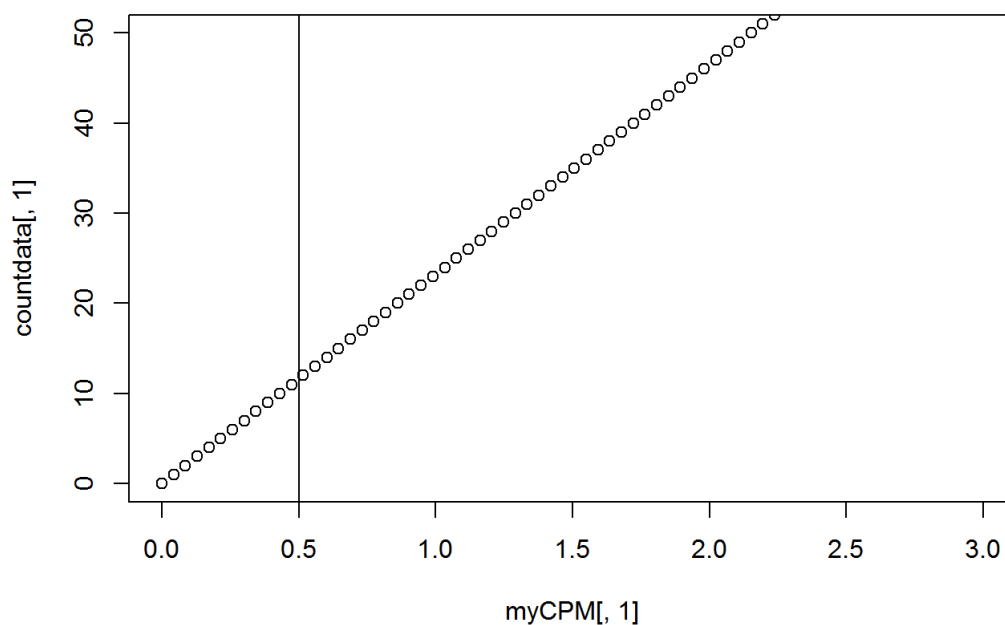
```



```

# Let us limit the x and y-axis so we can actually look to see what is happening at the smaller counts
plot(myCPM[,1],countdata[,1],ylim=c(0,50),xlim=c(0,3))
# Add a vertical line at 0.5 CPM
abline(v=0.5)

```

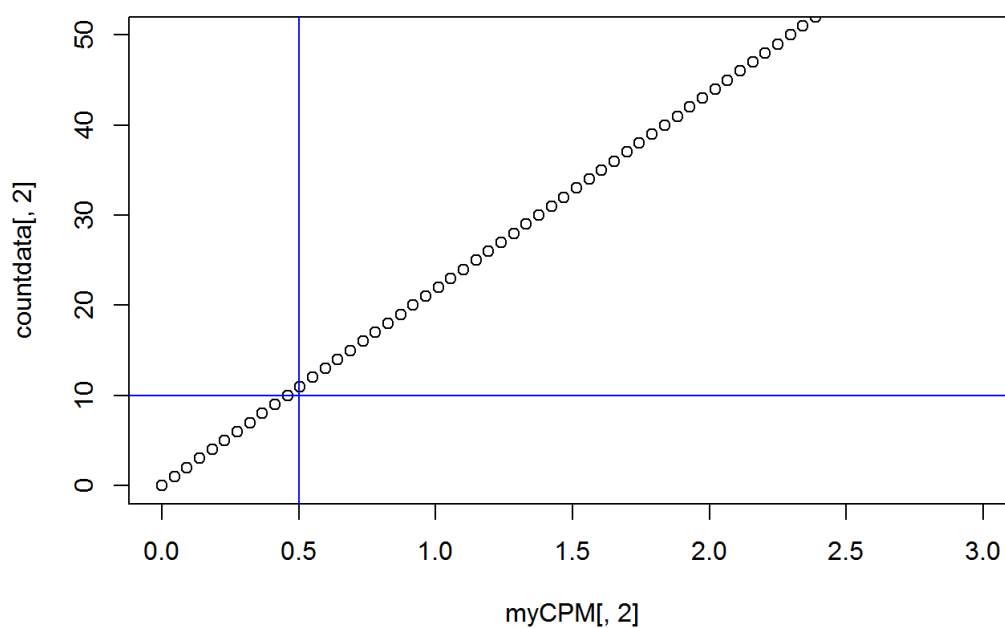


Challenge

1. Plot the counts-per-million versus counts for the second sample.
2. Add a vertical line at 0.5 and a horizontal line at 10.
3. Add the lines again, colouring them blue

HINT: use the `col` parameter.

Solution



Note: When in doubt, a threshold of 1 CPM in at least *minimum* group sample size is a good rule of thumb.

Convert counts to DGEList object

Next we'll create a `DGEList` object. This is an object used by *edgeR* to store count data. It has a number of slots for storing various parameters about the data.

```
y <- DGEList(counts.keep)
# have a look at y
y
```

An object of class "DGEList"

```
$counts
      MCL1.DG MCL1.DH MCL1.DI MCL1.DJ MCL1.DK MCL1.DL MCL1.LA MCL1.LB
497097      438      300       65      237      354      287        0        0
20671       106       182       82      105       43       82       16       25
27395       309       234      337      300      290      270      560      464
18777       652       515      948      935      928      791      826      862
21399      1604      1495     1721     1317     1159     1066     1334     1258

      MCL1.LC MCL1.LD MCL1.LE MCL1.LF
497097        0        0        0        0
20671         18         8         3        10
27395        489        328       307       342
18777        668        646       544       581
21399       1068        926       508       500
15799 more rows ...

$samples
      group lib.size norm.factors
MCL1.DG    1 23218026           1
MCL1.DH    1 21768136           1
MCL1.DI    1 24091588           1
MCL1.DJ    1 22656713           1
MCL1.DK    1 21522033           1
7 more rows ...
```

```
# See what slots are stored in y
names(y)
```

```
[1] "counts" "samples"
```

```
# Library size information is stored in the samples slot
y$samples
```

```
group lib.size norm.factors
MCL1.DG    1 23218026           1
MCL1.DH    1 21768136           1
MCL1.DI    1 24091588           1
MCL1.DJ    1 22656713           1
MCL1.DK    1 21522033           1
MCL1.DL    1 20008326           1
MCL1.LA    1 20384562           1
MCL1.LB    1 21698793           1
MCL1.LC    1 22235847           1
MCL1.LD    1 21982745           1
MCL1.LE    1 24719697           1
MCL1.LF    1 24652963           1
```

Quality control

Now that we have got rid of the lowly expressed genes and have our counts stored in a `DGEList` object, we can look at a few different plots to check that the data is good quality, and that the samples are as we would expect.

Library sizes and distribution plots

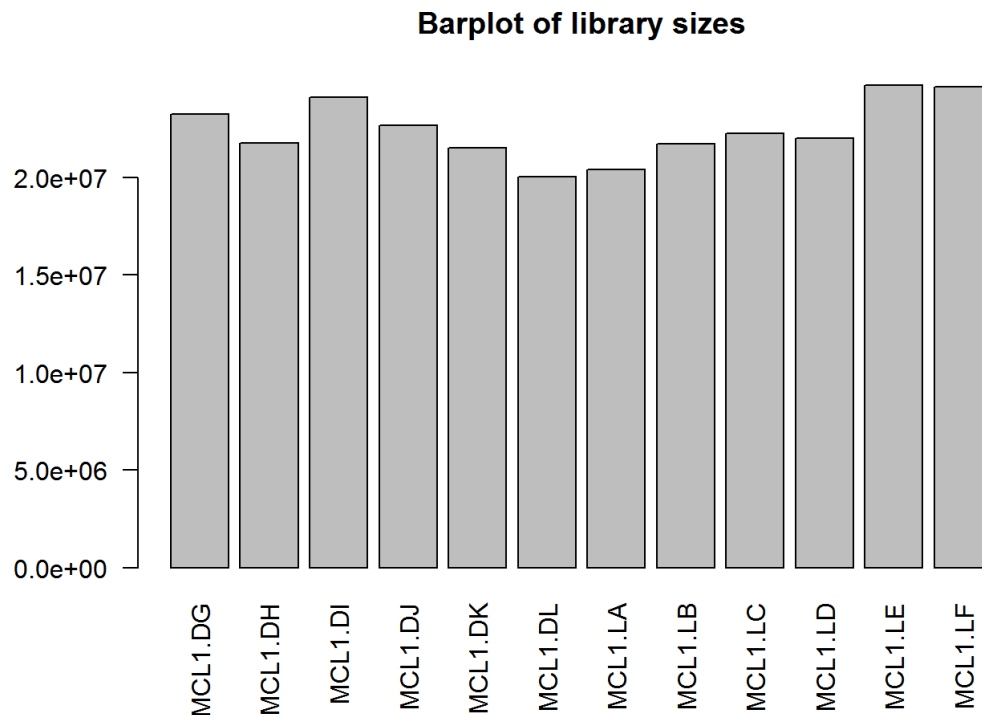
First, we can check how many reads we have for each sample in the `y`.

```
y$samples$lib.size
```

```
[1] 23218026 21768136 24091588 22656713 21522033 20008326 20384562
[8] 21698793 22235847 21982745 24719697 24652963
```

We can also plot the library sizes as a barplot to see whether there are any major discrepancies between the samples more easily.

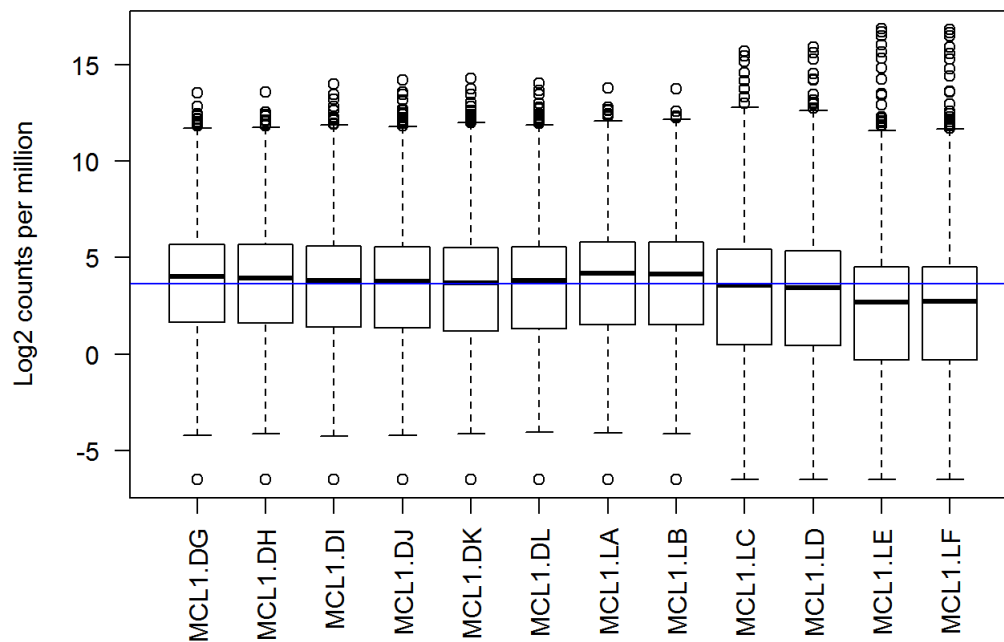
```
# The names argument tells the barplot to use the sample names on the x-axis
# The las argument rotates the axis names
barplot(y$samples$lib.size, names=colnames(y), las=2)
# Add a title to the plot
title("Barplot of library sizes")
```



Count data is not normally distributed, so if we want to examine the distributions of the raw counts we need to log the counts. Next we'll use box plots to check the distribution of the read counts on the log2 scale. We can use the `cpm` function to get log2 counts per million, which are corrected for the different library sizes. The `cpm` function also adds a small offset to avoid taking log of zero.

```
# Get log2 counts per million
logcounts <- cpm(y, log=TRUE)
# Check distributions of samples using boxplots
boxplot(logcounts, xlab="", ylab="Log2 counts per million", las=2)
# Let's add a blue horizontal line that corresponds to the median logCPM
abline(h=median(logcounts), col="blue")
title("Boxplots of logCPMs (unnormalised)")
```

Boxplots of logCPMs (unnormalised)



From the boxplots we see that overall the density distributions of raw log-intensities are not identical but still not very different. If a sample is really far above or below the blue horizontal line we may need to investigate that sample further.

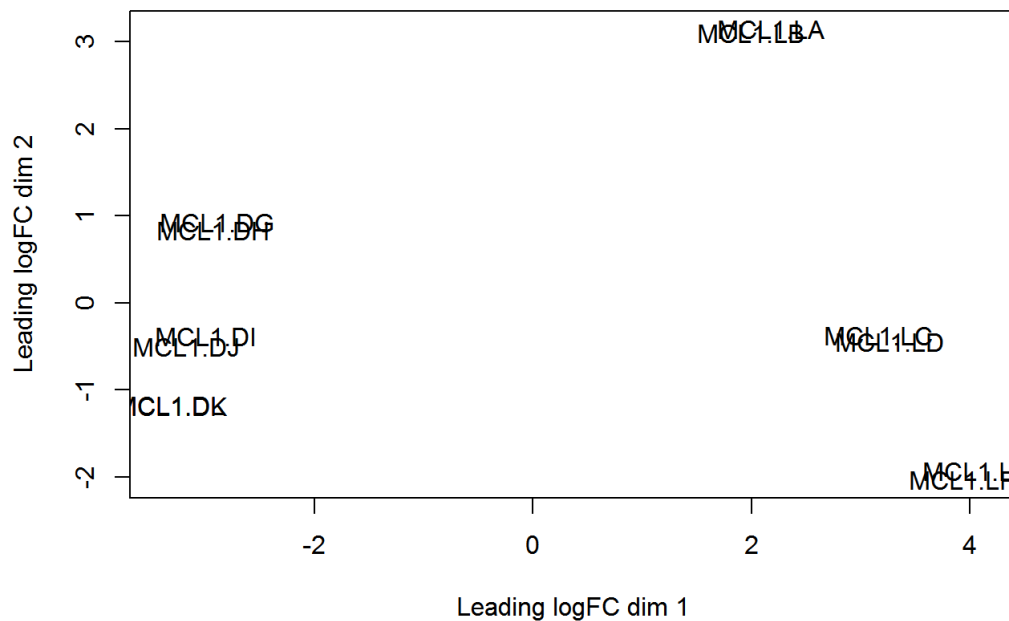
Discussion

Do any samples appear to be different compared to the others?

Multidimensional scaling plots

By far, one of the most important plots we make when we analyse RNA-Seq data are MDSplots. An MDSplot is a visualisation of a principle components analysis, which determines the greatest sources of variation in the data. A principle components analysis is an example of an unsupervised analysis, where we don't need to specify the groups. If your experiment is well controlled and has worked well, what we hope to see is that the greatest sources of variation in the data are the treatments/groups we are interested in. It is also an incredibly useful tool for quality control and checking for outliers. We can use the `plotMDS` function to create the MDS plot.

```
plotMDS(y)
```



It is a bit difficult to see exactly what is going on with the default plot, although we do see samples grouping together in pairs. To make this plot more informative, we can colour the samples according to the grouping information. We can also change the labels, or instead of labels we can have points.

```
# We specify the option to let us plot two plots side-by-side
par(mfrow=c(1,2))
# Let's set up colour schemes for CellType
# How many cell types and in what order are they stored?
levels(sampleinfo$CellType)
```

```
[1] "basal"  "luminal"
```

```
## Let's choose purple for basal and orange for luminal
col.cell <- c("purple","orange")[sampleinfo$CellType]
data.frame(sampleinfo$CellType,col.cell)
```

sampleinfo.CellType	col.cell
1	luminal orange
2	basal purple
3	basal purple
4	basal purple
5	basal purple
6	basal purple
7	basal purple
8	luminal orange
9	luminal orange
10	luminal orange
11	luminal orange
12	luminal orange

```
# Redo the MDS with cell type colouring
plotMDS(y,col=col.cell)
# Let's add a legend to the plot so we know which colours correspond to which cell type
legend("topleft",fill=c("purple","orange"),legend=levels(sampleinfo$CellType))
# Add a title
title("Cell type")

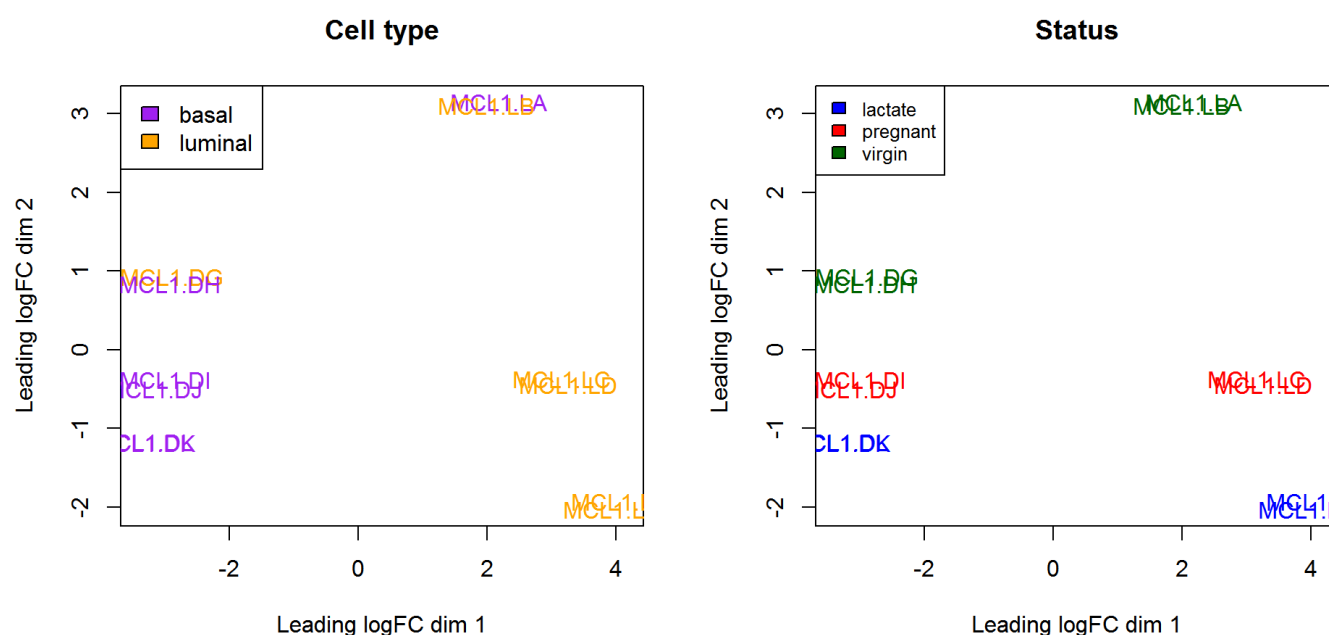
# Similarly for status
levels(sampleinfo$Status)
```

```
[1] "lactate" "pregnant" "virgin"
```

```
col.status <- c("blue","red","dark green")[sampleinfo$Status]
col.status
```

```
[1] "dark green" "dark green" "red"          "red"          "blue"
[6] "blue"       "dark green" "dark green" "red"          "red"
[11] "blue"       "blue"
```

```
plotMDS(y,col=col.status)
  legend("topleft",fill=c("blue","red","dark green"),legend=levels(sampleinfo$Status),cex=0.8)
  title("Status")
```



Discussion

Look at the MDS plot coloured by cell type. Is there something strange going on with the samples? Identify the two samples that don't appear to be in the right place.

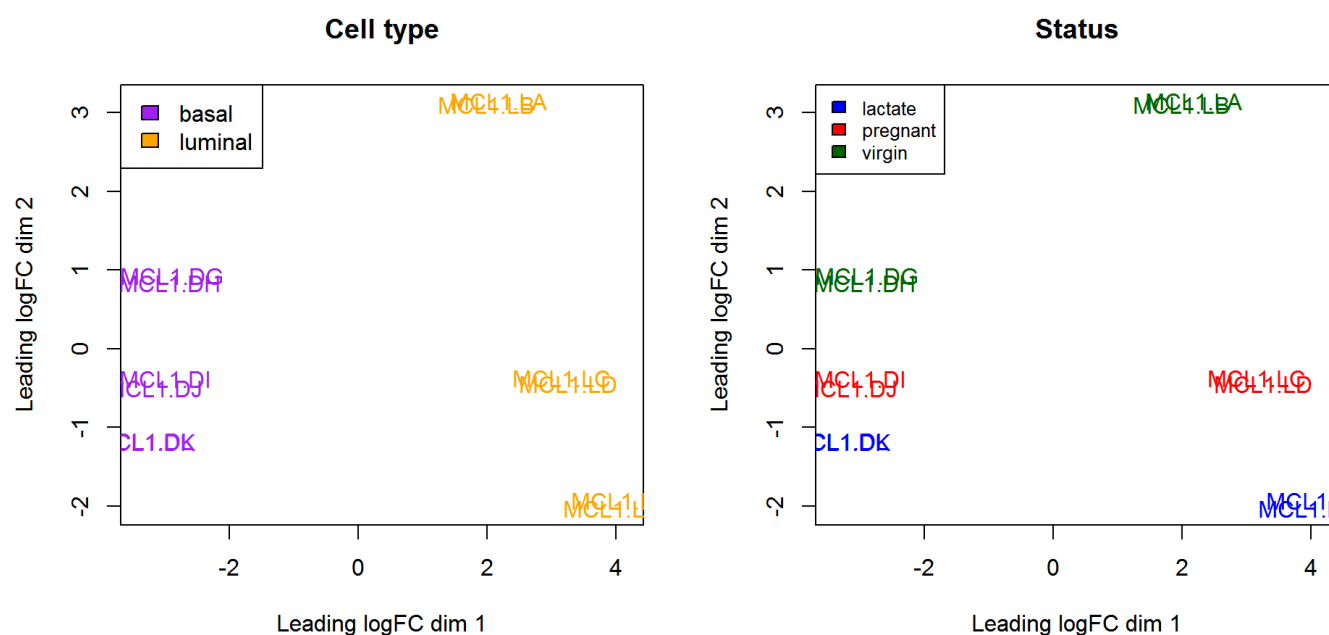
```
# There is a sample info corrected file in your data directory
# Old sampleinfo
sampleinfo
```

	FileName	SampleName	CellType	Status
1	MCL1.DG_BC2CTUACXX_ACTTGA_L002_R1	MCL1.DG	luminal	virgin
2	MCL1.DH_BC2CTUACXX_CAGATC_L002_R1	MCL1.DH	basal	virgin
3	MCL1.DI_BC2CTUACXX_ACAGTG_L002_R1	MCL1.DI	basal	pregnant
4	MCL1.DJ_BC2CTUACXX_CGATGT_L002_R1	MCL1.DJ	basal	pregnant
5	MCL1.DK_BC2CTUACXX_TTAGGC_L002_R1	MCL1.DK	basal	lactate
6	MCL1.DL_BC2CTUACXX_ATCACG_L002_R1	MCL1.DL	basal	lactate
7	MCL1.LA_BC2CTUACXX_GATCAG_L001_R1	MCL1.LA	basal	virgin
8	MCL1.LB_BC2CTUACXX_TGACCA_L001_R1	MCL1.LB	luminal	virgin
9	MCL1.LC_BC2CTUACXX_GCCAAT_L001_R1	MCL1.LC	luminal	pregnant
10	MCL1.LD_BC2CTUACXX_GGCTAC_L001_R1	MCL1.LD	luminal	pregnant
11	MCL1.LE_BC2CTUACXX_TAGCTT_L001_R1	MCL1.LE	luminal	lactate
12	MCL1.LF_BC2CTUACXX_CTTGTA_L001_R1	MCL1.LF	luminal	lactate

```
# I'm going to write over the sampleinfo object with the corrected sample info
sampleinfo <- read.delim("data/SampleInfo_Corrected.txt")
sampleinfo
```

	FileName	SampleName	CellType	Status
1	MCL1.DG_BC2CTUACXX_ACTTGA_L002_R1	MCL1.DG	basal	virgin
2	MCL1.DH_BC2CTUACXX_CAGATC_L002_R1	MCL1.DH	basal	virgin
3	MCL1.DI_BC2CTUACXX_ACAGTG_L002_R1	MCL1.DI	basal	pregnant
4	MCL1.DJ_BC2CTUACXX_CGATGT_L002_R1	MCL1.DJ	basal	pregnant
5	MCL1.DK_BC2CTUACXX_TTAGGC_L002_R1	MCL1.DK	basal	lactate
6	MCL1.DL_BC2CTUACXX_ATCACG_L002_R1	MCL1.DL	basal	lactate
7	MCL1.LA_BC2CTUACXX_GATCAG_L001_R1	MCL1.LA	luminal	virgin
8	MCL1.LB_BC2CTUACXX_TGACCA_L001_R1	MCL1.LB	luminal	virgin
9	MCL1.LC_BC2CTUACXX_GCCAAT_L001_R1	MCL1.LC	luminal	pregnant
10	MCL1.LD_BC2CTUACXX_GGCTAC_L001_R1	MCL1.LD	luminal	pregnant
11	MCL1.LE_BC2CTUACXX_TAGCTT_L001_R1	MCL1.LE	luminal	lactate
12	MCL1.LF_BC2CTUACXX_CTTGTA_L001_R1	MCL1.LF	luminal	lactate

```
# Redo the MDSplot with corrected information
par(mfrow=c(1,2))
col.cell <- c("purple","orange")[sampleinfo$CellType]
col.status <- c("blue","red","dark green")[sampleinfo$Status]
plotMDS(y,col=col.cell)
legend("topleft",fill=c("purple","orange"),legend=levels(sampleinfo$CellType))
title("Cell type")
plotMDS(y,col=col.status)
legend("topleft",fill=c("blue","red","dark green"),legend=levels(sampleinfo$Status),cex=0.8)
title("Status")
```



Discussion

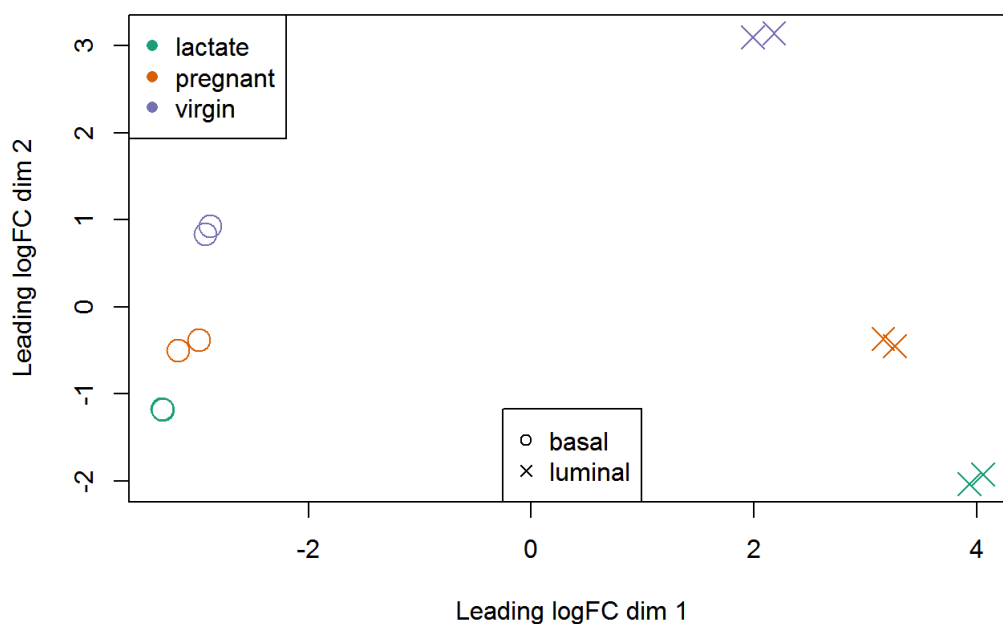
What is the greatest source of variation in the data (i.e. what does dimension 1 represent)? What is the second greatest source of variation in the data?

Challenge

1. Redo the plots choosing your own colours.
2. Change the plotting character to a symbol instead of the column names HINT: use `pch` argument. Try `pch=16` and see what happens.
3. Change the plotting characters such that basal samples have the value `1` and luminal samples have the value `4`.
4. Colour by status (lactate, pregnant, virgin)

Solution

```
[1] "#1B9E77" "#D95F02" "#7570B3"
```



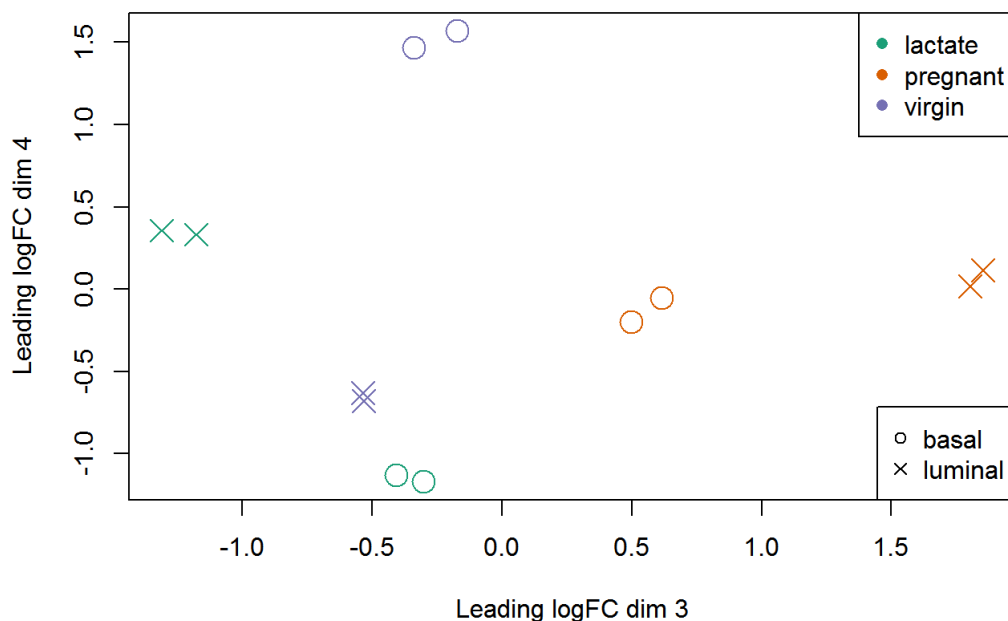
The distance between each pair of samples in the MDS plot is calculated as the leading fold change, defined as the root-mean-square of the largest 500 log₂-fold changes between that pair of samples. Replicate samples from the same group cluster together in the plot, while samples from different groups form separate clusters. This indicates that the differences between groups are larger than those within groups, i.e., differential expression is greater than the variance and can be detected. In the MDS plot, the distance between basal samples on the left and luminal cells on the right is about 6 units, corresponding to a leading fold change of about 64-fold ($2^6 = 64$) between basal and luminal. The expression differences between virgin, pregnant and lactating are greater for luminal cells than for basal.

Notes

- The MDS plot can be simply generated with `plotMDS(y)`. The additional code is purely for aesthetics, to improve the visualization of the groups.
- Clustering in the MDS plot can be used to motivate changes to the design matrix in light of potential batch effects. For example, imagine that the first replicate of each group was prepared at a separate time from the second replicate. If the MDS plot showed separation of samples by time, it might be worthwhile including time in the downstream analysis to account for the time-based effect.

`plotMDS` plots the first two dimensions as a default, however you can plot higher dimensions using the `dim` argument.

```
# Dimension 3 appears to separate pregnant samples from the rest. Dim4?
plotMDS(y,dim=c(3,4),col=col.status,pch=char.celltype,cex=2)
legend("topright",legend=levels(sampleinfo$Status),col=cols,pch=16)
legend("bottomright",legend=levels(sampleinfo$CellType),pch=c(1,4))
```



Another alternative is to generate an interactive MDS plot using the *Glimma* package. This allows the user to interactively explore the different dimensions.

```
labels <- paste(sampleinfo$SampleName, sampleinfo$CellType, sampleinfo$Status)
group <- paste(sampleinfo$CellType, sampleinfo$Status, sep=".")
group <- factor(group)
glMDSPlot(y, labels=labels, groups=group, folder="mds")
```

Glimma was created to make interactive versions of some of the popular plots from the *limma* package. At present it can be used to obtain MDS plots and mean-difference (MD) plots, which will be covered later. The output of `glMDSPlot` is an html page (`/mds/MDS-Plot.html`) that shows the MDS plot on the left, and the amount of variation explained by each dimension in a barplot on the right. The user can hover over points to find out sample information, and switch between successive dimensions in the MDS plot by clicking on the bars in the barplot. The default MDS plots shows dimensions 1 and 2.

Hierarchical clustering with heatmaps

An alternative to `plotMDS` for examining relationships between samples is using hierarchical clustering. Heatmaps are a nice visualisation to examine hierarchical clustering of your samples. We can do this using the `heatmap.2` function from the *gplots* package. In this example `heatmap.2` calculates a matrix of euclidean distances from the logCPM (`logcounts` object) for the 500 most variable genes. (Note this has more complicated code than plotting principle components using `plotMDS`.)

The *RColorBrewer* package has nicer colour schemes, accessed using the `brewer.pal` function. “RdYIBu” is a common choice, and “Spectral” is also nice.

Note: The `png` function will create a png file to save the plots created straight after, and will close this file when `dev.off()` is called. To see your plots interactively, simply omit those two lines.

Let's select data for the 500 most variable genes and plot the heatmap

```
# We estimate the variance for each row in the logcounts matrix
var_genes <- apply(logcounts, 1, var)
head(var_genes)
```

```
497097      20671      27395      18777      21399      58175
27.4382296  3.3141188  0.1596456  0.1313426  0.3947091  6.2321017
```

```
# Get the gene names for the top 500 most variable genes
select_var <- names(sort(var_genes, decreasing=TRUE))[1:500]
head(select_var)
```

```
[1] "16846" "72902" "56747" "497097" "24117" "109205"
```

```
# Subset logcounts matrix
highly_variable_lcpm <- logcounts[select_var,]
dim(highly_variable_lcpm)
```

```
[1] 500 12
```

```
head(highly_variable_lcpm)
```

```

MCL1.DG MCL1.DH MCL1.DI MCL1.DJ MCL1.DK MCL1.DL MCL1.LA
16846 0.4290261 1.394877 4.025241 6.068229 5.923462 5.467369 -6.486175
72902 1.4460253 2.569794 6.084854 5.183325 5.216547 4.629406 -6.486175
56747 3.3347872 2.975754 5.071799 5.016300 5.599113 6.048402 -6.486175
497097 4.2384664 3.785840 1.437863 3.388414 4.040841 3.843500 -6.486175
24117 7.8373931 7.899107 8.634338 8.582153 6.703626 6.776308 -1.710778
109205 5.5238723 5.717292 6.197664 6.203120 6.526628 6.529395 -6.486175

MCL1.LB MCL1.LC MCL1.LD MCL1.LE MCL1.LF
16846 -4.1268214 -4.155152 -6.48617500 -6.486175 -6.486175
72902 -6.4861750 -3.306148 -4.14190372 -6.486175 -6.486175
56747 -6.4861750 -3.306148 -6.48617500 -4.276248 -4.273190
497097 -6.4861750 -6.486175 -6.48617500 -6.486175 -6.486175
24117 -0.7124885 -0.366455 0.01712352 -4.276248 -4.273190
109205 -1.7975234 -2.388018 -3.29144547 -4.276248 -2.527764

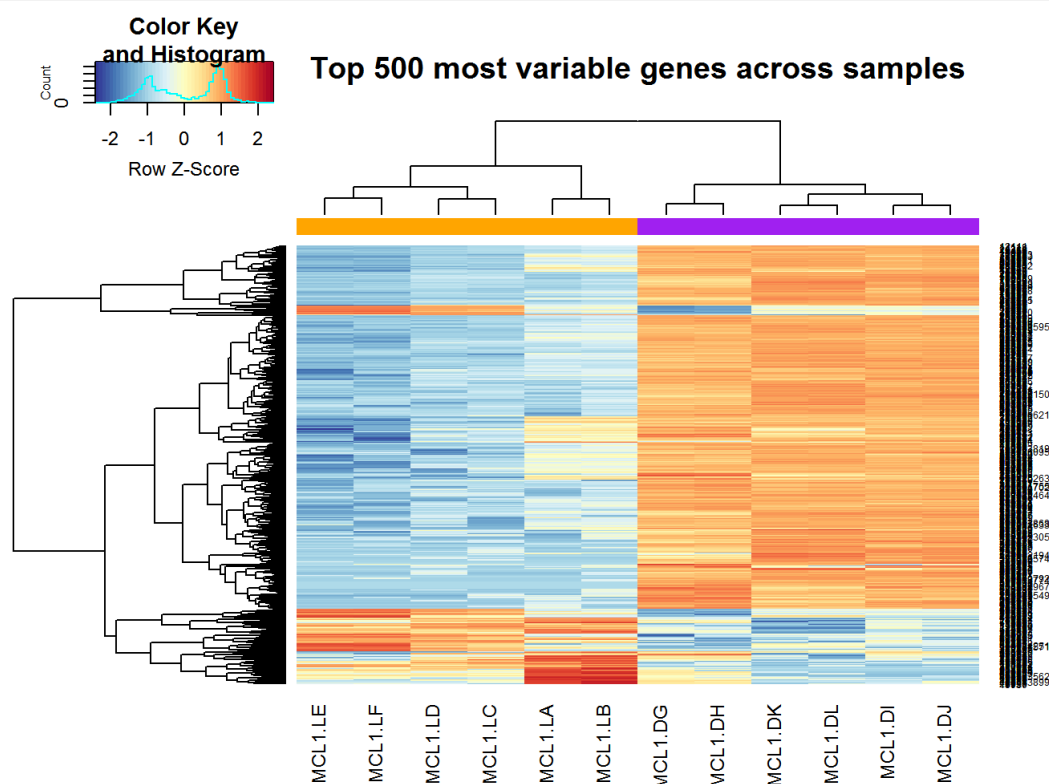
```

```

## Get some nicer colours
mypalette <- brewer.pal(11,"RdYlBu")
morecols <- colorRampPalette(mypalette)
# Set up colour vector for celltype variable
col.cell <- c("purple","orange")[sampleinfo$CellType]

# Plot the heatmap
heatmap.2(highly_variable_lcpm,col=rev(morecols(50)),trace="none", main="Top 500 most variable genes across
samples",ColSideColors=col.cell,scale="row")

```



```
# Save the heatmap
png(file="High_var_genes.heatmap.png")
heatmap.2(highly_variable_lcpm,col=rev(morecols(50)),trace="none", main="Top 500 most variable genes across
samples",ColSideColors=col.cell,scale="row")
dev.off()
```

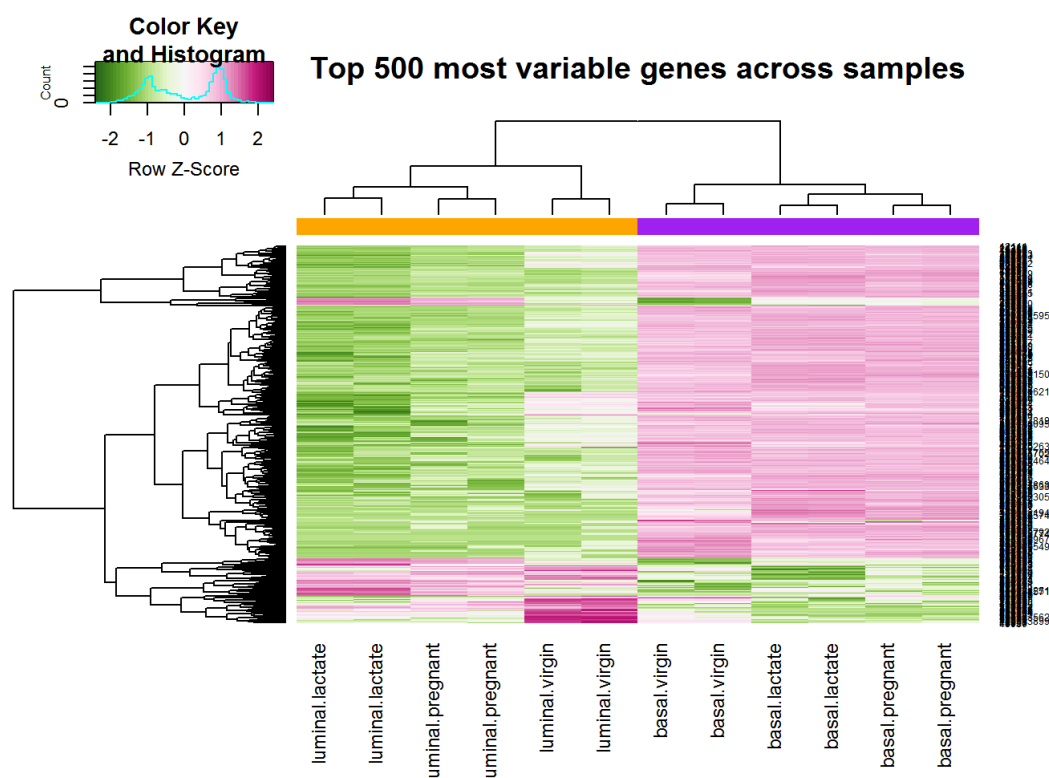
png

2

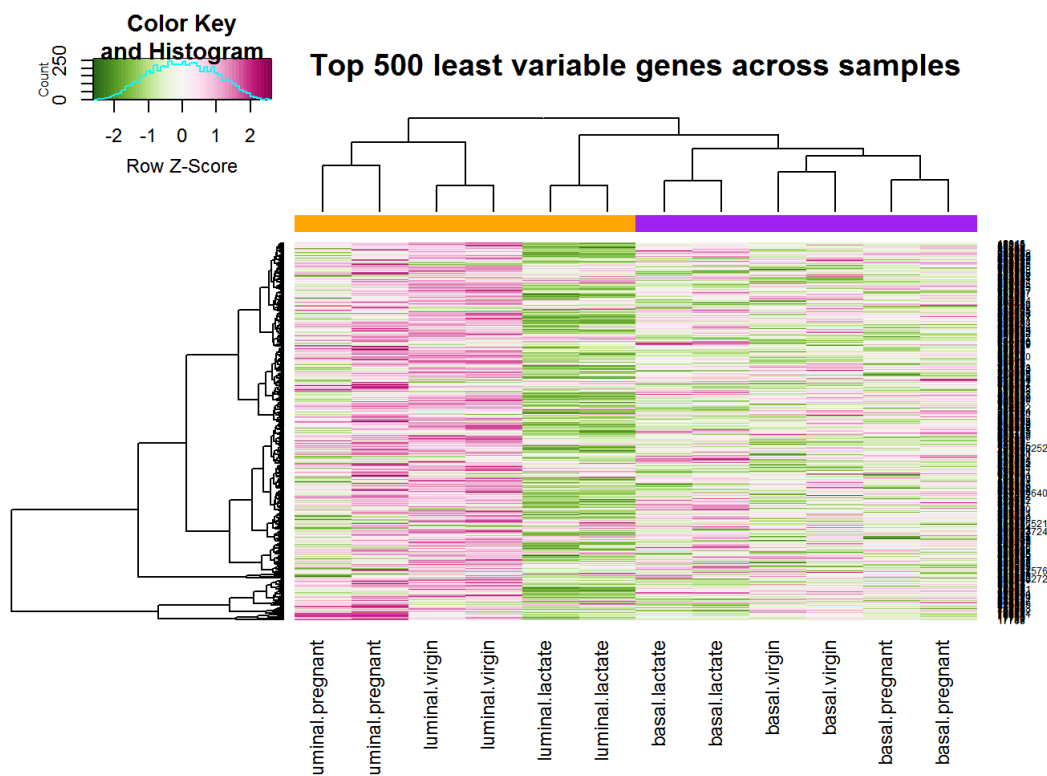
Challenge

1. Change the colour scheme to "PiYG" and redo the heatmap. Try `?RColorBrewer` and see what other colour schemes are available.
2. Change the sample names to `group` using the `labCol` argument
3. Redo the heatmap using the top 500 LEAST variable genes.

Solution



```
[1] "320714" "93684" "245638" "26433" "93747" "236792"
```



Normalisation for composition bias

TMM normalization is performed to eliminate composition biases between libraries (Mark D Robinson and Oshlack 2010). This generates a set of normalization factors, where the product of these factors and the library sizes defines the effective library size. The `calcNormFactors` function calculates the normalization factors between libraries. TMM normalisation (and most scaling normalisation methods) scale relative to one sample.

```
# Apply normalisation to DGEList object
y <- calcNormFactors(y)
```

This will update the normalisation factors in the `DGEList` object (their default values are 1). Take a look at the normalisation factors for these samples.

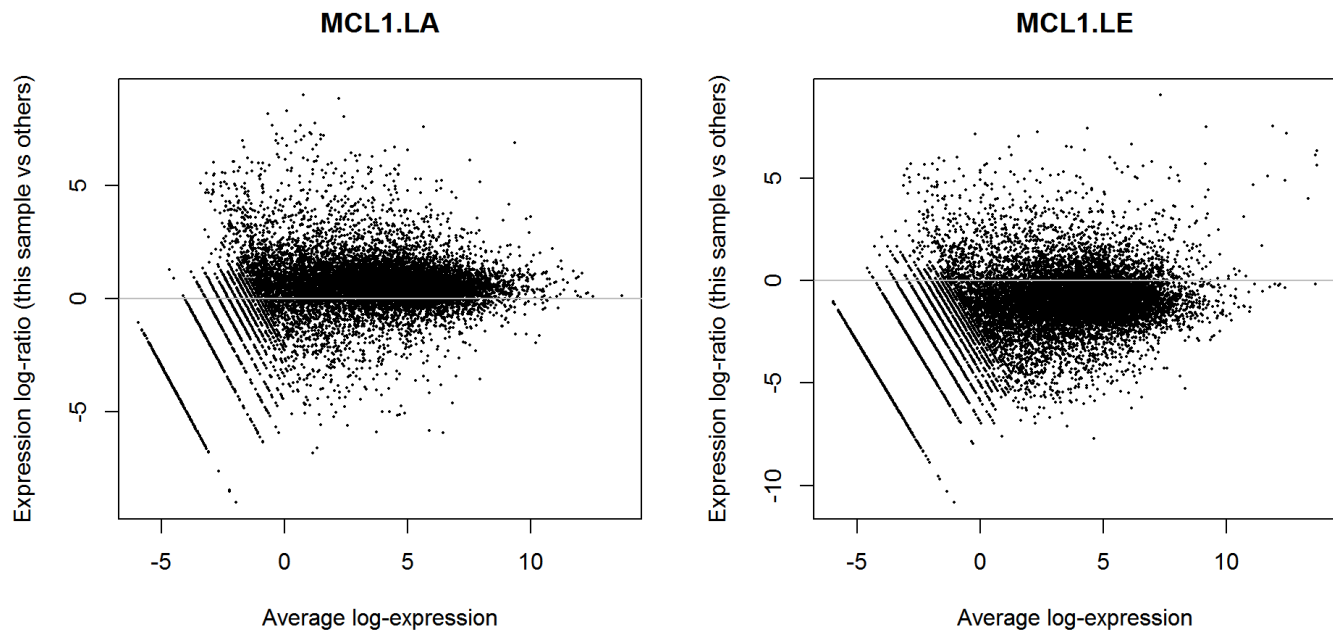
```
y$samples
```

group	lib.size	norm.factors
MCL1.DG	1 23218026	1.2368993
MCL1.DH	1 21768136	1.2139485
MCL1.DI	1 24091588	1.1255640
MCL1.DJ	1 22656713	1.0698261
MCL1.DK	1 21522033	1.0359212
MCL1.DL	1 20008326	1.0872153
MCL1.LA	1 20384562	1.3684449
MCL1.LB	1 21698793	1.3653200
MCL1.LC	1 22235847	1.0047431
MCL1.LD	1 21982745	0.9232822
MCL1.LE	1 24719697	0.5291015
MCL1.LF	1 24652963	0.5354877

The normalization factors multiply to unity across all libraries. A normalization factor below one indicates that the library size will be scaled down, as there is more suppression (i.e., composition bias) in that library relative to the other libraries. This is also equivalent to scaling the counts upwards in that sample. Conversely, a factor above one scales up the library size and is equivalent to downscaling the counts.

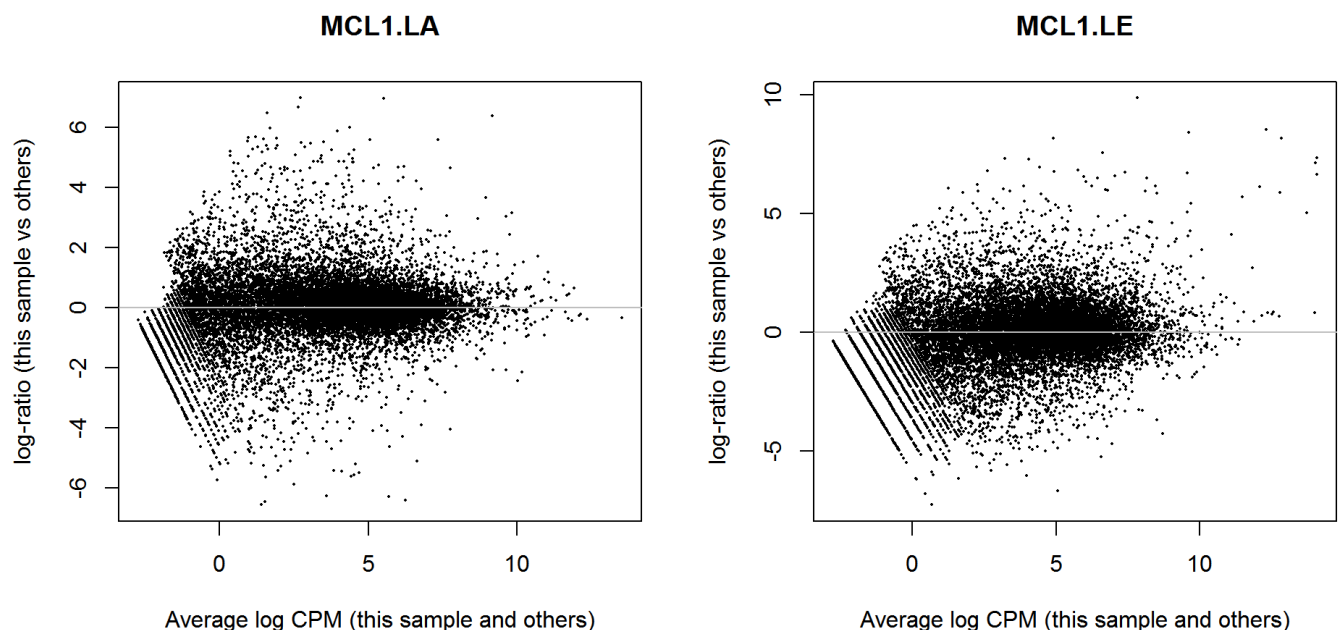
The last two samples have much smaller normalisation factors, and MCL1.LA and MCL1.LB have the largest. If we plot mean difference plots using the `plotMD` function for these samples, we should be able to see the composition bias problem. We will use the `logcounts`, which have been normalised for library size, but not for composition bias.

```
par(mfrow=c(1,2))
plotMD(logcounts,column = 7)
abline(h=0,col="grey")
plotMD(logcounts,column = 11)
abline(h=0,col="grey")
```



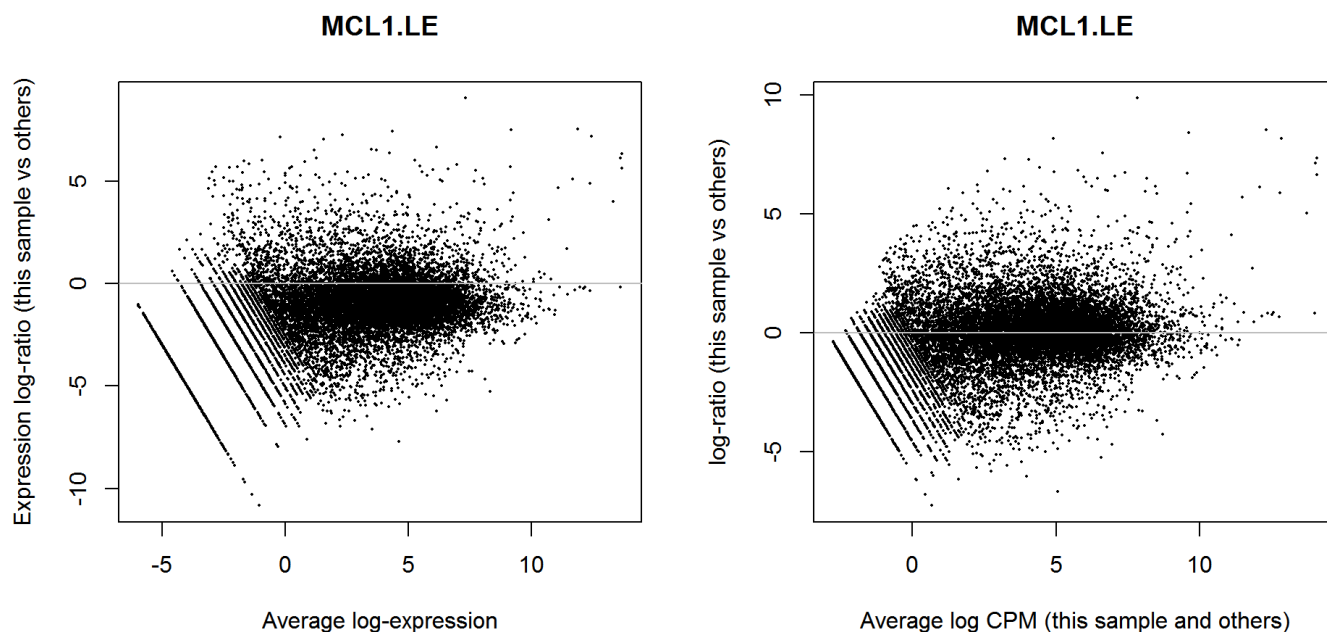
The mean-difference plots show average expression (mean: x-axis) against log-fold-changes (difference: y-axis). Because our `DGEList` object contains the normalisation factors, if we redo these plots using `y`, we should see the composition bias problem has been solved.

```
par(mfrow=c(1,2))
plotMD(y,column = 7)
abline(h=0,col="grey")
plotMD(y,column = 11)
abline(h=0,col="grey")
```



Challenge

Plot the biased and unbiased MD plots side by side for the same sample to see the before and after TMM normalisation effect.

Solution

We need to save a few data objects to use for Day 2 so we don't have to rerun everything

```
# save(group,y,logcounts,sampleinfo,file="day1objects.Rdata")
```

Differential expression with limma-voom

Now that we are happy that we have normalised the data and that the quality looks good, we can continue to testing for differentially expressed genes. There are a number of packages to analyse RNA-Seq data. The *limma* package (Ritchie et al. 2015) (since version 3.16.0) offers the `voom` function, which transforms the read counts into logCPMs while taking into account the mean-variance relationship in the data (C. W. Law et al. 2014). After vooming, users can apply a linear model to the voom transformed data to test for differentially expressed genes, using standard *limma* commands.

Load the objects into the workspace that we created yesterday

```
# load("day1objects.Rdata")
# objects()
```

Create the design matrix

First we need to create a design matrix for the groups (see the excellent *limma* user guide (<https://bioconductor.org/packages/release/bioc/vignettes/limma/inst/doc/usersguide.pdf>) for more information on design matrices). There are many different ways to set up your design matrix, and it is dictated by what comparisons you would like to test. We will follow the set-up from pg 43 of the *limma* vignette ("Interaction models: 2X2 factorial designs").

In this analysis let's assume that we will be testing differences in status in the different cell types separately. For example, we want to know which genes are differentially expressed between pregnant and lactating in basal cells only. We have previously coded the `group` variable, which is a concatenation of cell type and status. Coding the cell type and status in this way allows us to be flexible in specifying which comparisons we are interested in.

```
# Look at group variable again
group
```

```
[1] basal.virgin    basal.virgin    basal.pregnant  basal.pregnant
[5] basal.lactate   basal.lactate   luminal.virgin  luminal.virgin
[9] luminal.pregnant luminal.pregnant luminal.lactate luminal.lactate
6 Levels: basal.lactate basal.pregnant basal.virgin ... luminal.virgin
```

```
# Specify a design matrix without an intercept term
design <- model.matrix(~ 0 + group)
design
```

```
groupbasal.lactate groupbasal.pregnant groupbasal.virgin
 1              0              0              1
 2              0              0              1
 3              0              1              0
 4              0              1              0
 5              1              0              0
 6              1              0              0
 7              0              0              0
 8              0              0              0
 9              0              0              0
10              0              0              0
11              0              0              0
12              0              0              0
 groupluminal.lactate groupluminal.pregnant groupluminal.virgin
 1              0              0              0
 2              0              0              0
 3              0              0              0
 4              0              0              0
 5              0              0              0
 6              0              0              0
 7              0              0              1
 8              0              0              1
 9              0              1              0
10              0              1              0
11              1              0              0
12              1              0              0
attr(,"assign")
[1] 1 1 1 1 1 1
attr(,"contrasts")
attr(,"contrasts")$group
[1] "contr.treatment"
```

```
## Make the column names of the design matrix a bit nicer
colnames(design) <- levels(group)
design
```



```

basal.lactate basal.pregnant basal.virgin luminal.lactate
 1           0           0           1           0
 2           0           0           1           0
 3           0           1           0           0
 4           0           1           0           0
 5           1           0           0           0
 6           1           0           0           0
 7           0           0           0           0
 8           0           0           0           0
 9           0           0           0           0
10           0           0           0           0
11           0           0           0           1
12           0           0           0           1

luminal.pregnant luminal.virgin
 1           0           0
 2           0           0
 3           0           0
 4           0           0
 5           0           0
 6           0           0
 7           0           1
 8           0           1
 9           1           0
10           1           0
11           0           0
12           0           0

attr("assign")
[1] 1 1 1 1 1 1
attr("contrasts")
attr(,"contrasts")$group
[1] "contr.treatment"

```

Each column of the design matrix tells us which samples correspond to each group. The samples which come from basal cells from a lactating mouse correspond to columns 5 and 6 in the counts matrix, i.e. the samples which have 1s.

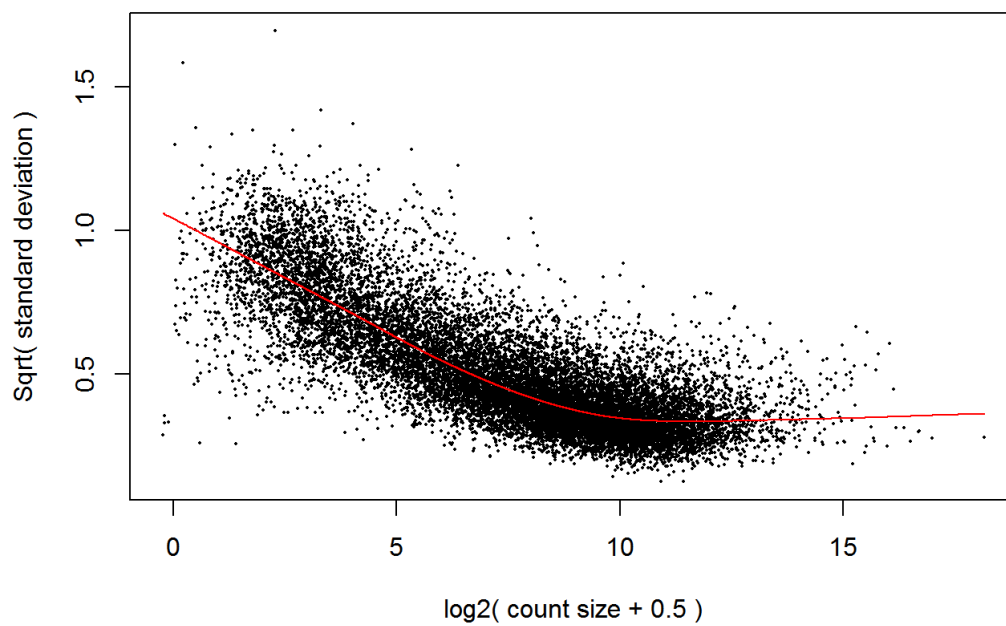
Voom transform the data

Once we have our design matrix ready to go, we can perform our voom transformation. Voom will automatically adjust the library sizes using the `norm.factors` already calculated. The voom transformation uses the experiment design matrix, and produces an `EList` object. We can add `plot=TRUE` to generate a plot of the mean-variance trend. This plot can also tell us if there are any genes that look really variable in our data, and if we've filtered the low counts adequately.

```

par(mfrow=c(1,1))
v <- voom(y,design,plot = TRUE)

```

voom: Mean-variance trend

The voom normalised log2 counts can be found in `v$E`. Take a look at what is in the voom object.

`v`

An object of class "EList"

\$targets

	group	lib.size	norm.factors
MCL1.DG	1	28718360	1.236899
MCL1.DH	1	26425396	1.213949
MCL1.DI	1	27116624	1.125564
MCL1.DJ	1	24238743	1.069826
MCL1.DK	1	22295130	1.035921

7 more rows ...

\$E

	MCL1.DG	MCL1.DH	MCL1.DI	MCL1.DJ	MCL1.DK	MCL1.DL	MCL1.LA
497097	3.932532	3.507368	1.272317	3.292541	3.9909851	3.724252	-5.8019424
20671	1.890808	2.787899	1.605217	2.121856	0.9642868	1.923156	-0.7575483
27395	3.429894	3.149591	3.637638	3.631978	3.7037376	3.636318	4.3286281
18777	4.505933	4.285975	5.128398	5.270351	5.3801014	5.185279	4.8889286
21399	5.804007	5.822559	5.988345	5.764344	5.7006304	5.615502	5.5801412

	MCL1.LB	MCL1.LC	MCL1.LD	MCL1.LE	MCL1.LF
497097	-5.8887821	-5.4816421	-5.343143	-4.709206	-4.7226146
20671	-0.2163568	-0.2721888	-1.255680	-1.901851	-0.3302972
27395	3.9707527	4.4535229	4.016606	4.555237	4.6973455
18777	4.8635986	4.9031416	4.993363	5.379583	5.4610208
21399	5.4087074	5.5797290	5.512504	5.280898	5.2446116

15799 more rows ...

\$weights

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
[1,]	42.85032	40.59638	19.78119	18.11020	38.62541	37.98661	0.7907909
[2,]	21.34610	19.99539	15.74020	14.40027	10.70068	10.49511	4.5401043
[3,]	35.05544	33.04583	39.72941	36.82081	35.31518	34.70612	50.4586968
[4,]	56.26175	53.86441	69.16366	66.65108	66.03797	65.46069	64.6692219
[5,]	76.92857	75.98600	76.79528	75.45282	71.62644	71.16558	73.5980084

	[,8]	[,9]	[,10]	[,11]	[,12]
[1,]	0.7907909	0.7907909	0.7907909	0.7907909	0.7907909
[2,]	4.7399033	3.4277768	3.2224692	2.1314506	2.1428929
[3,]	52.1833006	45.7806579	43.1302387	38.6112610	38.8559918
[4,]	66.0984998	60.0161723	57.3174922	54.0693981	54.3410792
[5,]	74.5613544	70.1642210	68.1238307	50.8570652	51.1210163

15799 more rows ...

\$design

	basal.lactate	basal.pregnant	basal.virgin	luminal.lactate
1	0	0	1	0
2	0	0	1	0
3	0	1	0	0
4	0	1	0	0
5	1	0	0	0

	luminal.pregnant	luminal.virgin
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0

7 more rows ...

What is contained in this object?

names(v)

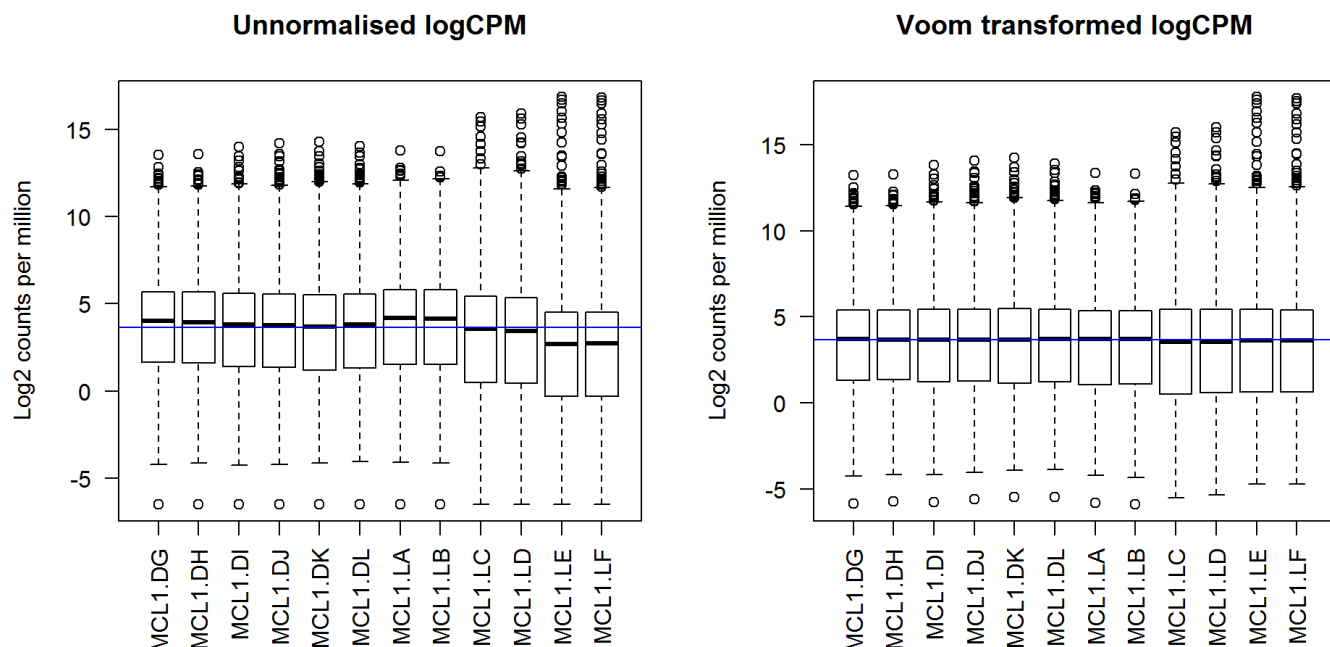
[1] "targets" "E" "weights" "design"

Challenge

1. What is in the `targets` slot of `v` and what does it correspond to in `y` ?
2. What are the dimensions of the `weights` slot in `v` ?

We can repeat the box plots for the normalised data to compare to before normalisation. The expression values in `v$E` are already log2 values so we don't need to log-transform.

```
par(mfrow=c(1,2))
boxplot(logcounts, xlab="", ylab="Log2 counts per million", las=2, main="Unnormalised logCPM")
## Let's add a blue horizontal line that corresponds to the median logCPM
abline(h=median(logcounts), col="blue")
boxplot(v$E, xlab="", ylab="Log2 counts per million", las=2, main="Voom transformed logCPM")
## Let's add a blue horizontal line that corresponds to the median logCPM
abline(h=median(v$E), col="blue")
```



Compare these box plots to the box plots we generated before performing the normalisation. Can you see any differences?

Testing for differential expression

Now that we have the voom transformed data we can use *limma* to test for differential expression. First we fit a linear model for each gene using the `lmFit` function in *limma*. `lmFit` needs the voom object and the design matrix that we have already specified, which is stored within the voom object.

```
# Fit the linear model
fit <- lmFit(v)
names(fit)
```

```
[1] "coefficients"      "stdev.unscaled"    "sigma"
[4] "df.residual"       "cov.coefficients"  "pivot"
[7] "rank"              "Amean"             "method"
[10] "design"
```

`lmFit` estimates group means according to the design matrix, as well as gene-wise variances. There are a number of items stored in the `fit` object, most of which are specific to the statistical testing, and we won't be discussing these in detail today.

Since we are interested in differences between groups, we need to specify which comparisons we want to test. The comparison of interest can be specified using the `makeContrasts` function. Here, we are interested in knowing which genes are differentially expressed between the pregnant and lactating group in the basal cells. This is done by defining the null hypothesis as `basal.pregnant - basal.lactate = 0` for each gene. Note that the group names must exactly match the column names of the design matrix.

```
cont.matrix <- makeContrasts(B.PregVsLac=basal.pregnant - basal.lactate, levels=design)
```

Take a look at the contrast matrix. The contrast matrix tells *limma* which columns of the design matrix we are interested in testing our comparison. Note that here we have specified only one comparison to test, but we can specify as many as we want in one go.

```
cont.matrix
```

Levels	Contrasts
	B.PregVsLac
basal.lactate	-1
basal.pregnant	1
basal.virgin	0
luminal.lactate	0
luminal.pregnant	0
luminal.virgin	0

Now we can apply the contrasts matrix to the `fit` object to get the statistics and estimated parameters of our comparison that we are interested in. Here we call the `contrasts.fit` function in *limma*.

```
fit.cont <- contrasts.fit(fit, cont.matrix)
```

The final step is to call the `eBayes` function, which performs empirical Bayes shrinkage on the variances, and estimates moderated t-statistics and the associated p-values.

```
fit.cont <- eBayes(fit.cont)
```

Check the dimensions of the fit object

```
dim(fit.cont)
```

```
[1] 15804      1
```

We can use the *limma* `decideTests` function to generate a quick summary of DE genes for the contrasts.

```
summa.fit <- decideTests(fit.cont)
summary(summa.fit)
```

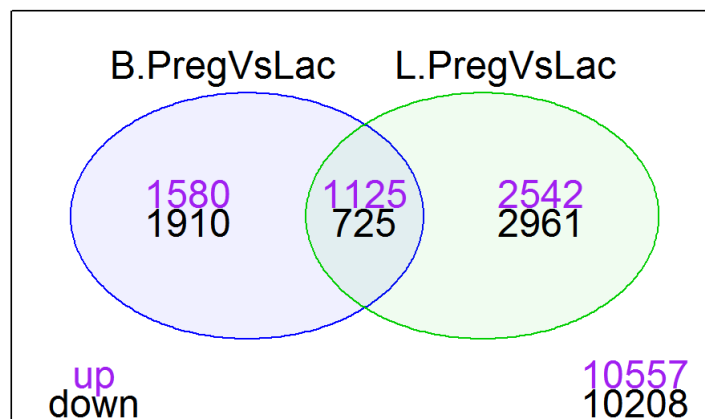
B.PregVsLac	
-1	2635
0	10464
1	2705

Challenge

1. Add another contrast to the contrasts matrix: `L.PregVsLac = luminal.pregnant - luminal.lactate` and re-run the code above. You should have two comparisons in `fit.cont` now.
2. Check out the `vennDiagram` function (HINT: type `?vennDiagram`). Can you show the overlap of differentially expressed genes between the two comparisons? How many genes are commonly differentially expressed?

Solution

B.PregVsLac	L.PregVsLac	
-1	2635	3686
0	10464	8451
1	2705	3667



The *limma* `topTable` function summarises the output in a table format. Significant DE genes for a particular comparison can be identified by selecting genes with a p-value smaller than a chosen cut-off value and/or a fold change greater than a chosen value in this table. By default the table will be sorted by the B statistic, which is the log-odds of differential expression. Usually the B statistic and p-value ranking will be the same, but this is not always the case. We will explicitly rank by p-value, which we can specify with the `sort.by` argument.

The `topTable` command will always output the top 10 genes by default, even if they are not statistically significant. We can specify the coefficient we are interested in by the name we used in the contrast matrix ("B.PregVsLac"), or by the column number.

```
topTable(fit.cont,coef="B.PregVsLac",sort.by="p")
```

	logFC	AveExpr	t	P.Value	adj.P.Val	B
24117	1.819943	2.975545	20.10780	1.063770e-10	1.016240e-06	14.96977
381290	-2.143885	3.944066	-19.07495	1.982934e-10	1.016240e-06	14.39556
78896	2.807548	3.036519	18.54773	2.758828e-10	1.016240e-06	14.07416
226101	-2.329744	6.223525	-18.26861	3.297667e-10	1.016240e-06	13.85802
16012	-2.896115	1.978449	-18.21525	3.413066e-10	1.016240e-06	13.46984
231830	2.253400	4.760597	18.02627	3.858161e-10	1.016240e-06	13.67600
16669	-2.312721	8.741892	-17.07937	7.264548e-10	1.640127e-06	13.17357
55987	-1.515469	2.834512	-16.64333	9.829870e-10	1.718703e-06	12.88782
231991	-2.598105	4.275930	-16.53634	1.059885e-09	1.718703e-06	12.66020
14620	3.600094	3.525281	16.46627	1.113755e-09	1.718703e-06	12.54071

```
## This will give the same output
topTable(fit.cont,coef=1,sort.by="p")
```

	logFC	AveExpr	t	P.Value	adj.P.Val	B
24117	1.819943	2.975545	20.10780	1.063770e-10	1.016240e-06	14.96977
381290	-2.143885	3.944066	-19.07495	1.982934e-10	1.016240e-06	14.39556
78896	2.807548	3.036519	18.54773	2.758828e-10	1.016240e-06	14.07416
226101	-2.329744	6.223525	-18.26861	3.297667e-10	1.016240e-06	13.85802
16012	-2.896115	1.978449	-18.21525	3.413066e-10	1.016240e-06	13.46984
231830	2.253400	4.760597	18.02627	3.858161e-10	1.016240e-06	13.67600
16669	-2.312721	8.741892	-17.07937	7.264548e-10	1.640127e-06	13.17357
55987	-1.515469	2.834512	-16.64333	9.829870e-10	1.718703e-06	12.88782
231991	-2.598105	4.275930	-16.53634	1.059885e-09	1.718703e-06	12.66020
14620	3.600094	3.525281	16.46627	1.113755e-09	1.718703e-06	12.54071

Adding annotation and saving the results

We have a list of significantly differentially expressed genes, but the only annotation we can see is the Entrez Gene ID, which is not very informative. We would like to add some annotation information. There are a number of ways to do this. We will demonstrate how to do this using the *org.Mm.eg.db* package.

First we need to decide what information we want. In order to see what we can extract we can run the `columns` function on the annotation database.

```
columns(org.Mm.eg.db)
```

```
[1] "ACCNUM"      "ALIAS"      "ENSEMBL"    "ENSEMBLPROT"
[5] "ENSEMBLTRANS" "ENTREZID"   "ENZYME"     "EVIDENCE"
[9] "EVIDENCEALL"  "GENENAME"   "GO"         "GOALL"
[13] "IPI"         "MGI"        "ONTOLOGY"   "ONTOLOGYALL"
[17] "PATH"        "PFAM"       "PMID"       "PROSITE"
[21] "REFSEQ"      "SYMBOL"     "UNIGENE"    "UNIPROT"
```

We definitely want gene symbols and perhaps the full gene name. Let's build up our annotation information in a separate data frame using the `select` function.

```
ann <- select(org.Mm.eg.db,keys=rownames(fit.cont),columns=c("ENTREZID","SYMBOL","GENENAME"))
```

'select()' returned 1:1 mapping between keys and columns

```
# Have a look at the annotation
head(ann)
```

	ENTREZID	SYMBOL	GENENAME
1	497097	Xkr4	X-linked Kx blood group related 4
2	20671	Sox17	SRY (sex determining region Y)-box 17
3	27395	Mrpl15	mitochondrial ribosomal protein L15
4	18777	Lypla1	lysophospholipase 1
5	21399	Tcea1	transcription elongation factor A (SII) 1
6	58175	Rgs20	regulator of G-protein signaling 20

Let's double check that the `ENTREZID` column matches exactly to our `fit.cont` rownames.

```
table(ann$ENTREZID==rownames(fit.cont))
```

```
TRUE
15804
```

We can slot in the annotation information into the `genes` slot of `fit.cont`. (Please note that if the `select` function returns a 1:many mapping then you can't just append the annotation to the fit object. An alternative way to get annotation will be discussed tomorrow during the analysis of the second dataset.)

```
fit.cont$genes <- ann
```

Now when we run the `topTable` command, the annotation information should be included in the output.

```
topTable(fit.cont,coef="B.PregVsLac",sort.by="p")
```

ENTREZID	SYMBOL		GENENAME			
24117	24117	Wif1	Wnt inhibitory factor 1			
381290	381290	Atp2b4	ATPase, Ca++ transporting, plasma membrane 4			
78896	78896	1500015010Rik	RIKEN cDNA 1500015010 gene			
226101	226101	Myof	myoferlin			
16012	16012	Igfbp6	insulin-like growth factor binding protein 6			
231830	231830	Micall2	MICAL-like 2			
16669	16669	Krt19	keratin 19			
55987	55987	Cpxm2	carboxypeptidase X 2 (M14 family)			
231991	231991	Creb5	cAMP responsive element binding protein 5			
14620	14620	Gjb3	gap junction protein, beta 3			
	logFC	AveExpr	t	P.Value	adj.P.Val	B
24117	1.819943	2.975545	20.10780	1.063770e-10	1.016240e-06	14.96977
381290	-2.143885	3.944066	-19.07495	1.982934e-10	1.016240e-06	14.39556
78896	2.807548	3.036519	18.54773	2.758828e-10	1.016240e-06	14.07416
226101	-2.329744	6.223525	-18.26861	3.297667e-10	1.016240e-06	13.85802
16012	-2.896115	1.978449	-18.21525	3.413066e-10	1.016240e-06	13.46984
231830	2.253400	4.760597	18.02627	3.858161e-10	1.016240e-06	13.67600
16669	-2.312721	8.741892	-17.07937	7.264548e-10	1.640127e-06	13.17357
55987	-1.515469	2.834512	-16.64333	9.829870e-10	1.718703e-06	12.88782
231991	-2.598105	4.275930	-16.53634	1.059885e-09	1.718703e-06	12.66020
14620	3.600094	3.525281	16.46627	1.113755e-09	1.718703e-06	12.54071

To get the full table (i.e. the information for all genes, not just the top 10) we can specify `n="Inf"` .

```
limma.res <- topTable(fit.cont,coef="B.PregVsLac",sort.by="p",n="Inf")
```

We can save the results table using the `write.csv` function, which writes the results out to a csv file, which you can open in excel.

```
write.csv(limma.res,file="B.PregVsLacResults.csv",row.names=FALSE)
```

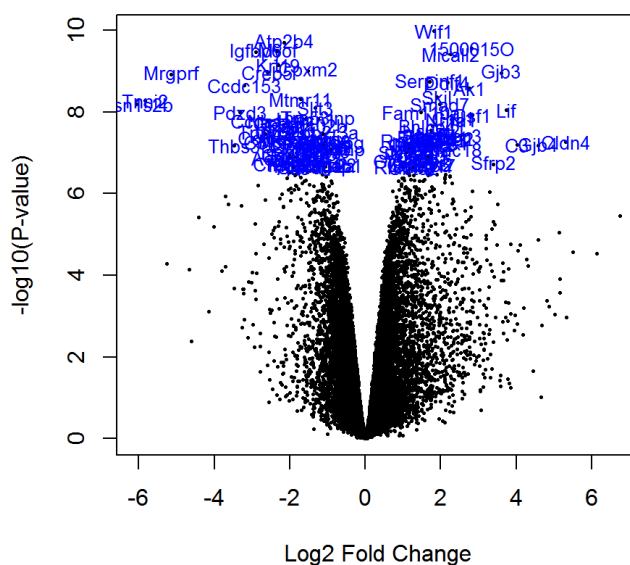
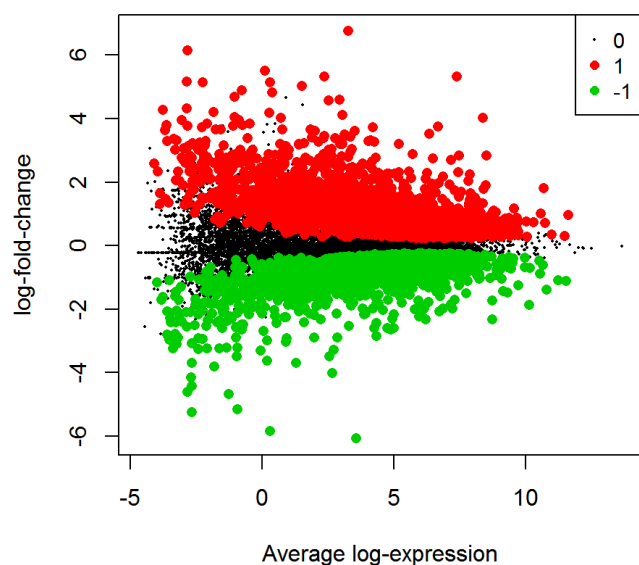
A note about deciding how many genes are significant: In order to decide which genes are differentially expressed, we usually take a cut-off of 0.05 on the adjusted p-value, NOT the raw p-value. This is because we are testing more than 15000 genes, and the chances of finding differentially expressed genes is very high when you do that many tests. Hence we need to control the false discovery rate, which is the adjusted p-value column in the results table. What this means is that if 100 genes are significant at a 5% false discovery rate, we are willing to accept that 5 will be false positives. Note that the `decideTests` function displays significant genes at 5% FDR.

Plots after testing for DE

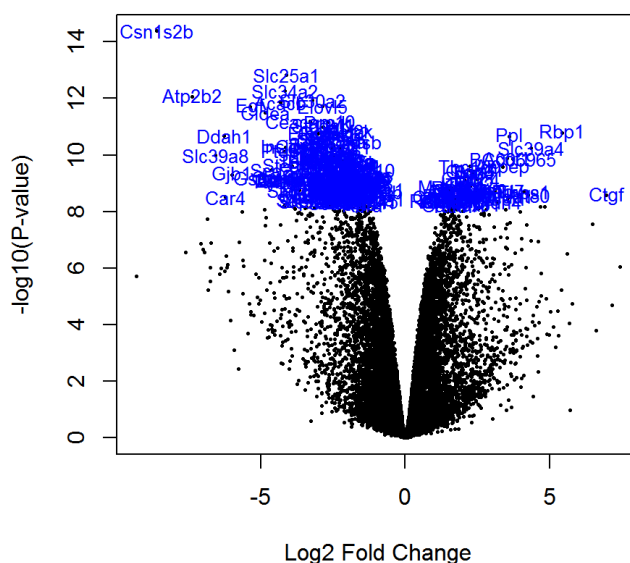
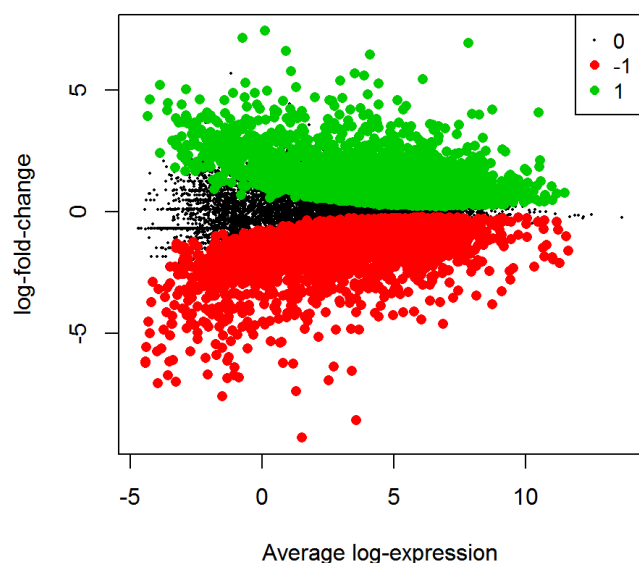
Let's do a few plots to make sure everything looks good and that we haven't made a mistake in the analysis. Genome-wide plots that are useful for checking are MAplots (or MDplots) and volcano plots. There are functions in limma for plotting these with `fit.cont` as input.

```
# We want to highlight the significant genes. We can get this from decideTests.
par(mfrow=c(1,2))
plotMD(fit.cont,coef=1,status=summa.fit[, "B.PregVsLac"])

# For the volcano plot we have to specify how many of the top genes to highlight.
# We can also specify that we want to plot the gene symbol for the highlighted genes.
# let's highlight the top 100 most DE genes
volcanoplot(fit.cont,coef=1,highlight=100,names=fit.cont$genes$SYMBOL)
```

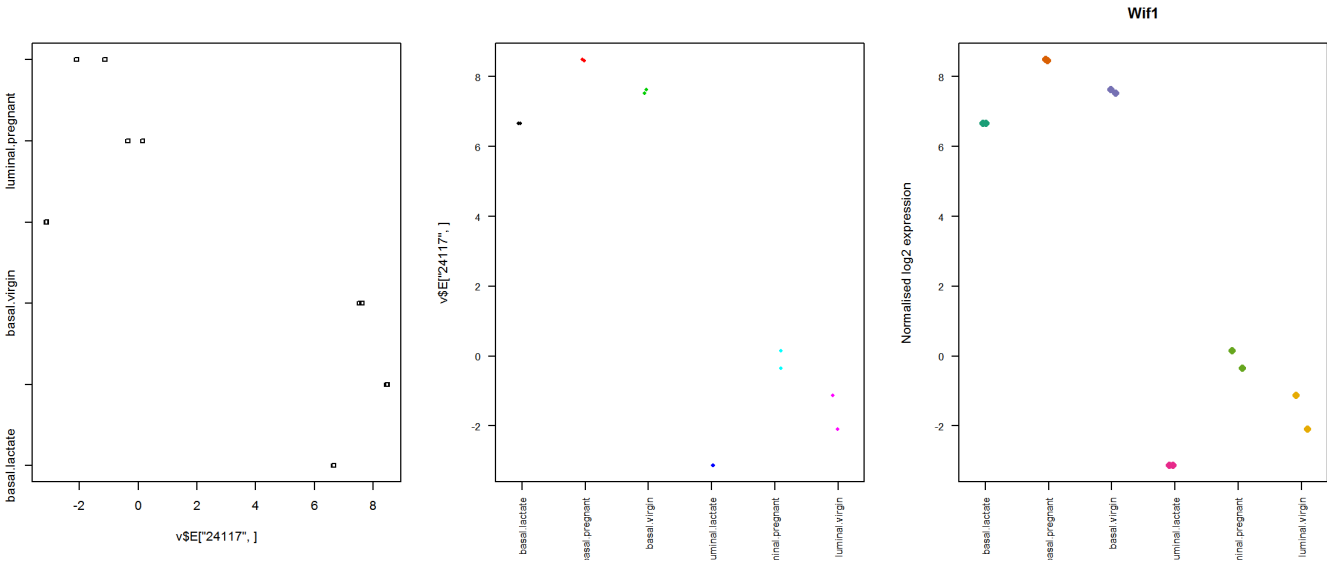

B.PregVsLac**Challenge**

Look at the MD plot and volcano plot for the second comparison, L.PregVsLac. Change the number of highlighted genes to 200 in the volcano plot.

L.PregVsLac

Before following up on the DE genes with further lab work, it is recommended to have a look at the expression levels of the individual samples for the genes of interest. We can quickly look at grouped expression using `stripchart`. We can use the normalised log expression values in the voom object (`v$E`).

```
par(mfrow=c(1,3))
# Let's look at the first gene in the topTable, Wif1, which has a rowname 24117
stripchart(v$E["24117"],~group)
# This plot is ugly, let's make it better
stripchart(v$E["24117"],~group,vertical=TRUE,las=2,cex.axis=0.8,pch=16,col=1:6,method="jitter")
# Let's use nicer colours
nice.col <- brewer.pal(6,name="Dark2")
stripchart(v$E["24117"],~group,vertical=TRUE,las=2,cex.axis=0.8,pch=16,cex=1.3,col=nice.col,method="jitter",
ylab="Normalised log2 expression",main="Wif1")
```



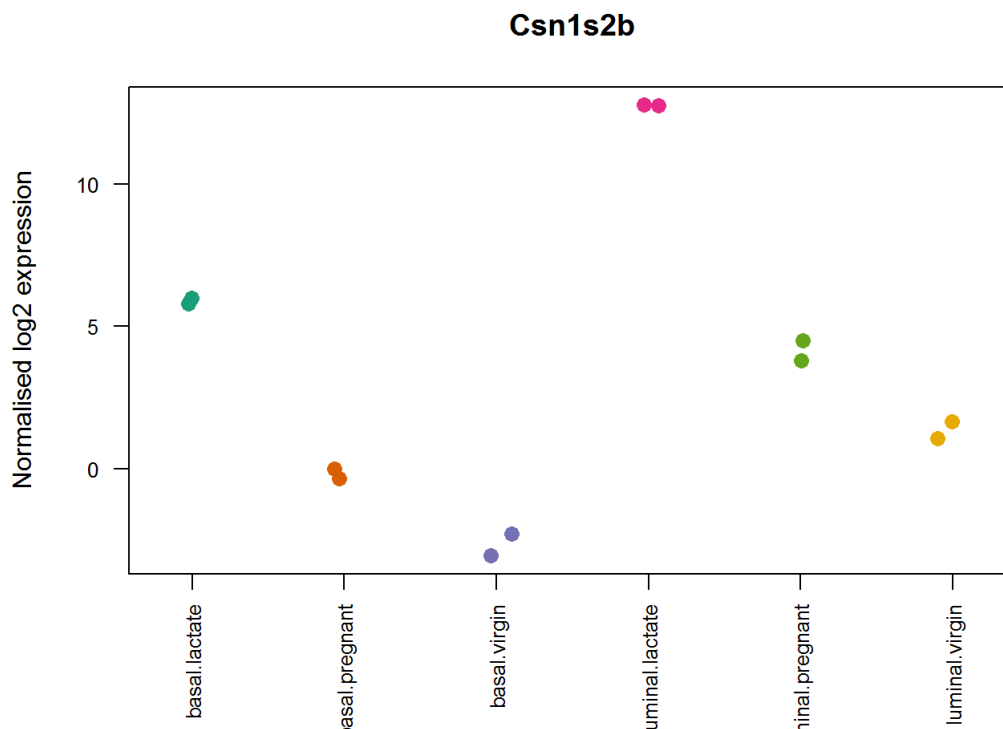
Notice anything interesting about the expression of this gene?

Challenge

Take the top gene from the L.PregVsLactate comparison and make a stripchart of grouped expression as above. (Don't forget to change the title of the plot.)

Solution

ENTREZID	SYMBOL	GENENAME				
12992	12992	Csn1s2b				
13358	13358	Slc25a1				
20531	20531	Slc34a2				
11941	11941	Atp2b2				
230810	230810	Slc30a2				
100705	100705	Acacb				
68801	68801	Elovl5				
13645	13645	Egf				
12683	12683	Cidea				
26366	26366	Ceacam10				
12992		casein alpha s2-like B				
13358	solute carrier family 25 (mitochondrial carrier, citrate transporter), member 1					
20531	solute carrier family 34 (sodium phosphate), member 2					
11941	ATPase, Ca++ transporting, plasma membrane 2					
230810	solute carrier family 30 (zinc transporter), member 2					
100705	acetyl-Coenzyme A carboxylase beta					
68801	ELOVL family member 5, elongation of long chain fatty acids (yeast)					
13645	epidermal growth factor					
12683	cell death-inducing DNA fragmentation factor, alpha subunit-like effector A					
26366	carcinoembryonic antigen-related cell adhesion molecule 10					
	logFC	AveExpr	t	P.Value	adj.P.Val	B
12992	-8.603611	3.5629500	-46.84744	4.172002e-15	6.593433e-11	23.84700
13358	-4.124175	5.7796989	-34.76897	1.526326e-13	1.206103e-09	21.38712
20531	-4.177812	4.2786290	-31.24354	5.524752e-13	2.910439e-09	20.23287
11941	-7.386986	1.2821431	-30.01476	8.946195e-13	3.534642e-09	18.54201
230810	-3.203118	2.6958115	-29.31342	1.188209e-12	3.654867e-09	19.42820
100705	-4.314320	4.4409137	-28.93689	1.387573e-12	3.654867e-09	19.30790
68801	-2.863304	6.4552045	-28.18823	1.900049e-12	3.828415e-09	19.08355
13645	-5.362664	0.7359047	-28.14183	1.937947e-12	3.828415e-09	17.22920
12683	-4.840655	3.3749575	-27.04271	3.123304e-12	5.484522e-09	18.58932
26366	-3.295621	1.8210142	-25.45612	6.437086e-12	1.005946e-08	17.82256



An interactive version of the volcano plot above that includes the raw per sample values in a separate panel is possible via the `glXYPLOT` function in the *Glimma* package.

```
group2 <- group
levels(group2) <- c("basal.lactate","basal.preg","basal.virgin","lum.lactate", "lum.preg", "lum.virgin")
glXYPLOT(x=fit.cont$coefficients[,1], y=fit.cont$lods[,1],
         xlab="logFC", ylab="B", main="B.PregVsLac",
         counts=y$counts, groups=group2, status=summa.fit[,1],
         anno=fit.cont$genes, side.main="ENTREZID", folder="volcano")
```

This function creates an html page (`./volcano/XY-Plot.html`) with a volcano plot on the left and a plot showing the log-CPM per sample for a selected gene on the right. A search bar is available to search for genes of interest.

Testing relative to a threshold (TREAT)

When there is a lot of differential expression, sometimes we may want to cut-off on a fold change threshold as well as a p-value threshold so that we follow up on the most biologically significant genes. However, it is not recommended to simply rank by p-value and then discard genes with small logFC's, as this has been shown to increase the false discovery rate. In other words, you are not controlling the false discovery rate at 5% anymore. There is a function called `treat` in the *limma* package that performs this style of analysis correctly (D. J. McCarthy and Smyth 2009). `treat` will simply take our `fit.cont` object, as well as a user-specified log fold change cut-off, and recalculate the moderated t-statistics and p-values with the new information about logFC.

```
# Let's decide that we are only interested in genes that have a absolute logFC of 1.
# This corresponds to a fold change of 2, or 0.5 (i.e. double or half).
# We can perform a treat analysis which ranks our genes according to p-value AND logFC.
# This is easy to do after our analysis, we just give the treat function the fit.cont object and specify our
cut-off.

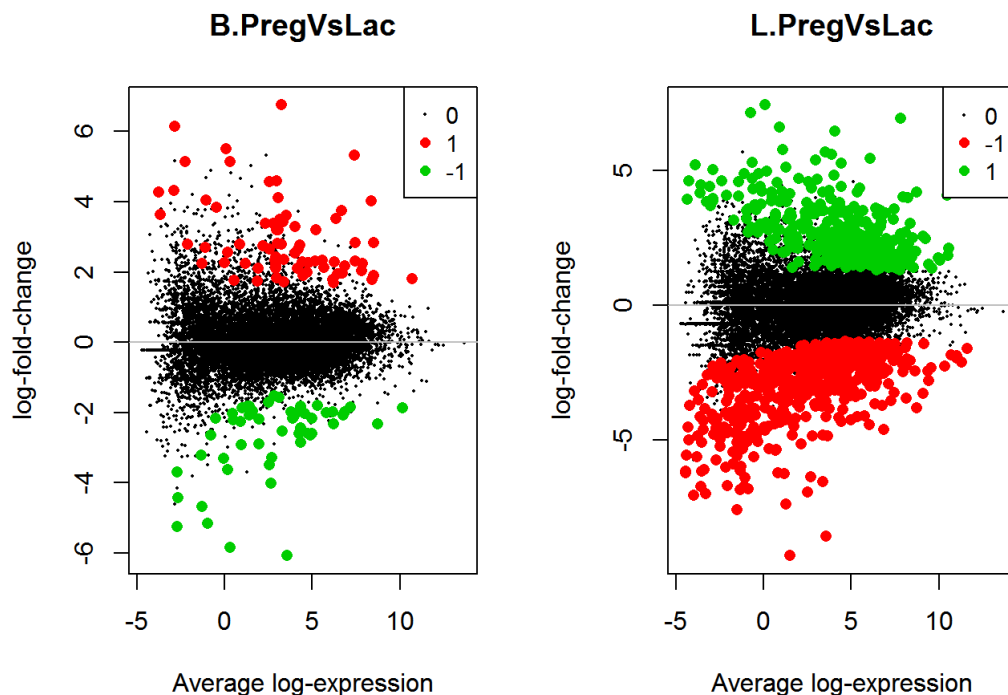
fit.treat <- treat(fit.cont,lfc=1)
res.treat <- decideTests(fit.treat)
summary(res.treat)
```

B.PregVsLac	L.PregVsLac	
-1	53	631
0	15677	14775
1	74	398

```
topTable(fit.treat,coef=1,sort.by="p")
```

ENTREZID	SYMBOL		GENENAME		
211577	211577	Mrgprf	MAS-related GPR, member F		
78896	78896	1500015010Rik	RIKEN cDNA 1500015010 gene		
16012	16012	Igfbp6	insulin-like growth factor binding protein 6		
21953	21953	Tnni2	troponin I, skeletal, fast 2		
14620	14620	Gjb3	gap junction protein, beta 3		
12992	12992	Csn1s2b	casein alpha s2-like B		
270150	270150	Ccdc153	coiled-coil domain containing 153		
226101	226101	Myof	myoferlin		
381290	381290	Atp2b4	ATPase, Ca++ transporting, plasma membrane 4		
231991	231991	Creb5	cAMP responsive element binding protein 5		
	logFC	AveExpr	t	P.Value	adj.P.Val
211577	-5.146268	-0.9368335	-13.18562	7.329299e-09	6.669358e-05
78896	2.807548	3.0365195	11.94135	2.234154e-08	6.669358e-05
16012	-2.896115	1.9784488	-11.92570	2.267536e-08	6.669358e-05
21953	-5.827889	0.3020716	-11.93183	2.295775e-08	6.669358e-05
14620	3.600094	3.5252805	11.89242	2.342636e-08	6.669358e-05
12992	-6.070143	3.5629500	-11.83199	2.532027e-08	6.669358e-05
270150	-3.211148	-1.3408388	-10.67393	7.828123e-08	1.767367e-04
226101	-2.329744	6.2235246	-10.42714	1.012983e-07	2.001148e-04
381290	-2.143885	3.9440659	-10.17758	1.322118e-07	2.050763e-04
231991	-2.598105	4.2759295	-10.17157	1.330809e-07	2.050763e-04

```
# Notice that much fewer genes are highlighted in the MAplot
par(mfrow=c(1,2))
plotMD(fit.treat,coef=1,status=res.treat[, "B.PregVsLac"])
abline(h=0,col="grey")
plotMD(fit.treat,coef=2,status=res.treat[, "L.PregVsLac"])
abline(h=0,col="grey")
```



Challenge

Change the cut-off so that we are interested in genes that change at least 50% on the fold change scale.

HINT: what is the corresponding logFC value of 50% fold change? Assume basal.pregnant is 50% higher than basal.lactate

Solution

B.PregVsLac	L.PregVsLac	
-1	234	1328
0	15250	13397
1	320	1079

ENTREZID	SYMBOL	GENENAME			
78896	78896 1500015010Rik	RIKEN cDNA 1500015010 gene			
16012	16012 Igfbp6	insulin-like growth factor binding protein 6			
211577	211577 Mrgprf	MAS-related GPR, member F			
381290	381290 Atp2b4	ATPase, Ca++ transporting, plasma membrane 4			
14620	14620 Gjb3	gap junction protein, beta 3			
226101	226101 Myof	myoferlin			
24117	24117 Wif1	Wnt inhibitory factor 1			
231830	231830 Micall2	MICAL-like 2			
21953	21953 Tnni2	troponin I, skeletal, fast 2			
231991	231991 Creb5	cAMP responsive element binding protein 5			
	logFC	AveExpr	t	P.Value	adj.P.Val
78896	2.807548	3.0365195	14.68324	2.122741e-09	1.107588e-05
16012	-2.896115	1.9784488	-14.53610	2.387463e-09	1.107588e-05
211577	-5.146268	-0.9368335	-14.50548	2.595874e-09	1.107588e-05
381290	-2.143885	3.9440659	-13.87032	4.068604e-09	1.107588e-05
14620	3.600094	3.5252805	13.79074	4.435571e-09	1.107588e-05
226101	-2.329744	6.2235246	-13.68164	4.766155e-09	1.107588e-05
24117	1.819943	2.9755445	13.64479	4.905791e-09	1.107588e-05
231830	2.253400	4.7605970	13.34681	6.327022e-09	1.249903e-05
21953	-5.827889	0.3020716	-12.95757	9.721628e-09	1.399330e-05
231991	-2.598105	4.2759295	-12.81319	1.010560e-08	1.399330e-05

An interactive version of the mean-difference plots is possible via the `gMDPlot` function in the *Glimma* package.

```
gMDPlot(fit.treat, coef=1, counts=y$counts, groups=group2,
        status=res.treat, id.column="ENTREZID", main="B.PregVsLac",
        folder="md")
```

As with the volcano plot example above, this function creates an html page (`./md/MD-Plot.html`) that allows the user to search for their favourite gene.

Gene Set Testing

Sometimes there is quite a long list of differentially expressed genes to interpret after a differential expression analysis, and it is usually infeasible to go through the list one gene at a time trying to understand its biological function. A common downstream procedure is gene set testing, which aims to understand which pathways/gene networks the differentially expressed genes are implicated in.

There are a number of different ways to go about testing for enrichment of biological pathways, and the test you choose usually depends on the question you're asking. There are two kinds of tests: self-contained and competitive gene set tests. Self-contained tests, which include the `ROAST` procedure, ask the question "Are the genes in the set/pathway differentially expressed as a whole?" Competitive gene set tests, like `goana` and `camera` ask the question whether the differentially expressed genes tend to be over-represented in the gene set, compared to all the other genes in the experiment. These different questions use different statistical methodology.

Gene ontology testing with goana

First, we will perform a gene ontology (GO) enrichment analysis using the `goana` function in *limma*. There are approximately 20,000 GO terms, and they are split into three categories: BP (biological process), MF (molecular function) and CC (cellular component). `goana` uses annotation from the appropriate Bioconductor package and can be used for any of the five species specified (Hs, Mm, Rn, Dm or Pt). `goana` has an advantage over other methods, such as DAVID, in that there is the option to take into account the gene length bias inherent in RNA-Seq data.

Suppose we want to identify GO terms that are over-represented in the basal lactating group compared to the basal pregnancy group. This can be achieved by applying the `goana` function to the differential expression results of that comparison. `goana` takes the `fit.cont` object, the coefficient of interest and the species. The top set of most enriched GO terms can be viewed with the `topGO` function.

```
go <- goana(fit.cont, coef="B.PregVsLac", species = "Mm")
topGO(go, n=10)
```

	Term	Ont	N	Up	Down	
G0:0003723	RNA binding	MF	1454	476	160	
G0:1990904	ribonucleoprotein complex	CC	742	293	63	
G0:0030529	intracellular ribonucleoprotein complex	CC	741	292	63	
G0:0022613	ribonucleoprotein complex biogenesis	BP	361	178	24	
G0:0042254	ribosome biogenesis	BP	244	139	7	
G0:0022626	cytosolic ribosome	CC	107	80	1	
G0:0006364	rRNA processing	BP	171	102	2	
G0:0016072	rRNA metabolic process	BP	197	111	8	
G0:0003676	nucleic acid binding	MF	2858	714	435	
G0:0032991	macromolecular complex	CC	4108	937	635	
	P.Up	P.Down				
G0:0003723	1.082317e-53	1.0000000				
G0:1990904	3.128474e-50	1.0000000				
G0:0030529	7.770671e-50	1.0000000				
G0:0022613	3.381031e-46	1.0000000				
G0:0042254	1.149588e-45	1.0000000				
G0:0022626	1.948117e-39	1.0000000				
G0:0006364	3.565529e-36	1.0000000				
G0:0016072	4.947163e-36	1.0000000				
G0:0003676	2.575720e-33	0.9999123				
G0:0032991	3.237272e-29	0.9999895				

The row names of the output are the universal identifiers of the GO terms, with one term per row. The Term column gives the names of the GO terms. These terms cover three domains - biological process (BP), cellular component (CC) and molecular function (MF), as shown in the Ont column. The N column represents the total number of genes that are annotated with each GO term. The Up and Down columns represent the number of differentially expressed genes that overlap with the genes in the GO term. The P.Up and P.Down columns contain the p-values for over-representation of the GO term across the set of up- and down-regulated genes, respectively. The output table is sorted by the minimum of P.Up and P.Down by default.

An additional refinement is to supply `goana` with the gene lengths using the `covariate` argument. In the original data matrix that we loaded into R, there is a column called "Length".

```
colnames(seqdata)
```

```
[1] "EntrezGeneID"
[2] "Length"
[3] "MCL1.DG_BC2CTUACXX_ACTTGA_L002_R1"
[4] "MCL1.DH_BC2CTUACXX_CAGATC_L002_R1"
[5] "MCL1.DI_BC2CTUACXX_ACAGTG_L002_R1"
[6] "MCL1.DJ_BC2CTUACXX_CGATGT_L002_R1"
[7] "MCL1.DK_BC2CTUACXX_TTAGGC_L002_R1"
[8] "MCL1.DL_BC2CTUACXX_ATCACG_L002_R1"
[9] "MCL1.LA_BC2CTUACXX_GATCAG_L001_R1"
[10] "MCL1.LB_BC2CTUACXX_TGACCA_L001_R1"
[11] "MCL1.LC_BC2CTUACXX_GCCAAT_L001_R1"
[12] "MCL1.LD_BC2CTUACXX_GGCTAC_L001_R1"
[13] "MCL1.LE_BC2CTUACXX_TAGCTT_L001_R1"
[14] "MCL1.LF_BC2CTUACXX_CTTGTA_L001_R1"
```

In order to get the gene lengths for every gene in `fit.cont`, we can use the `match` command. Note that the gene length supplied needs to be in the correct order.

```
m <- match(rownames(fit.cont), seqdata$EntrezGeneID)
gene_length <- seqdata$Length[m]
head(gene_length)
```

```
[1] 3634 3130 4203 2433 2847 2241
```

```
# Rerun goana with gene length information
go_length <- goana(fit.cont, coef="B.PregVsLac", species="Mm", covariate=go_length)
topGO(go_length, n=10)
```

	Term Ont	N	Up	Down	
G0:0003723	RNA binding	MF	1454	476	160
G0:1990904	ribonucleoprotein complex	CC	742	293	63
G0:0030529	intracellular ribonucleoprotein complex	CC	741	292	63
G0:0042254	ribosome biogenesis	BP	244	139	7
G0:0022613	ribonucleoprotein complex biogenesis	BP	361	178	24
G0:0022626	cytosolic ribosome	CC	107	80	1
G0:0006364	rRNA processing	BP	171	102	2
G0:0016072	rRNA metabolic process	BP	197	111	8
G0:0005840	ribosome	CC	212	107	5
G0:0022625	cytosolic large ribosomal subunit	CC	58	49	0
	P.Up	P.Down			
G0:0003723	8.494584e-55	1			
G0:1990904	8.278850e-54	1			
G0:0030529	2.003347e-53	1			
G0:0042254	1.651659e-48	1			
G0:0022613	5.780765e-48	1			
G0:0022626	4.702353e-44	1			
G0:0006364	3.030188e-38	1			
G0:0016072	7.851352e-38	1			
G0:0005840	4.133967e-34	1			
G0:0022625	2.140251e-32	1			

Challenge

Perform GO analysis for the second comparison, "L.PregVsLac", taking into account gene length information

Solution

	Term Ont	N	Up	Down	P.Up
G0:0005488	binding	MF	9690	2619	2217 8.445897e-42
G0:0005634	nucleus	CC	5548	1628	1207 2.036292e-36
G0:0003676	nucleic acid binding	MF	2858	938	538 7.282394e-36
G0:0005515	protein binding	MF	6737	1915	1488 4.110947e-35
G0:0044444	cytoplasmic part	CC	6646	1580	1902 3.145543e-01
G0:0010467	gene expression	BP	3948	1213	750 2.870675e-33
G0:0005739	mitochondrion	CC	1598	272	568 1.000000e+00
G0:1901363	heterocyclic compound binding	MF	4367	1314	935 2.189563e-31
G0:0003723	RNA binding	MF	1454	526	244 3.019330e-31
G0:0022626	cytosolic ribosome	CC	107	79	4 3.204763e-31
	P.Down				
G0:0005488	9.999998e-01				
G0:0005634	9.999999e-01				
G0:0003676	1.000000e+00				
G0:0005515	1.000000e+00				
G0:0044444	2.844972e-33				
G0:0010467	1.000000e+00				
G0:0005739	9.457323e-32				
G0:1901363	9.999999e-01				
G0:0003723	1.000000e+00				
G0:0022626	1.000000e+00				

Notes

- Users can specify the domain of the enriched GO terms in topGO. For instance, topGO(go,ontology="BP") lists the top set of most enriched GO terms that are related to a biological process. This avoids other domains that are not of interest.
- The goana function uses the NCBI RefSeq annotation. Therefore, the Entrez Gene identifier (ID) should be supplied for each gene as the row names of the fit object.
- Users should set species according to the organism being studied.

CAMERA gene set testing using the Broad's curated gene sets

Other databases of gene sets that are available come from the Broad Institute's Molecular Signatures Database (MSigDB (<http://software.broadinstitute.org/gsea/msigdb>)). CAMERA is good option for testing a very large number of gene sets such as the MSigDB sets, as it is very fast. CAMERA is known as a competitive gene set test, however it has the advantage that it can take into account inter-

gene correlation within each gene set (D. Wu and Smyth 2012). It also works seamlessly with a `voom` object, taking into account the mean-variance relationship in RNA-Seq data.

Here we will be using the C2 gene sets for mouse, available as `.rdata` files from the WEHI bioinformatics page <http://bioinf.wehi.edu.au/software/MSigDB/index.html> (<http://bioinf.wehi.edu.au/software/MSigDB/index.html>). The C2 gene sets contain 4725 curated gene sets collected from a variety of places: BioCarta, KEGG, Pathway Interaction Database, Reactome as well as some published studies. It doesn't include GO terms.

```
# Load in the mouse c2 gene sets
# The R object is called Mm.c2
load("data/mouse_c2_v5.rdata")
# Have a look at the first few gene sets
names(Mm.c2)[1:5]
```

```
[1] "KEGG_GLYCOLYSIS_GLUONEOGENESIS"
[2] "KEGG_CITRATE_CYCLE_TCA_CYCLE"
[3] "KEGG_PENTOSE_PHOSPHATE_PATHWAY"
[4] "KEGG_PENTOSE_AND_GLUCURONATE_INTERCONVERSIONS"
[5] "KEGG_FRUCTOSE_AND_MANNOSE_METABOLISM"
```

```
# Number of gene sets in C2
length(Mm.c2)
```

```
[1] 4725
```

The gene identifiers are Entrez Gene ID, which is the same as the rownames of our `voom` object. We need to map the Entrez gene ids between the list of gene sets and our `voom` object. We can do this using the `ids2indices` function.

```
c2.ind <- ids2indices(Mm.c2, rownames(v))
```

CAMERA takes as input the `voom` object `v`, the indexed list of gene sets `c2.ind`, the design matrix, the contrast being tested, as well as some other arguments. By default, CAMERA can estimate the correlation for each gene set separately. However, in practise, it works well to set a small inter-gene correlation of about 0.05 using the `inter.gene.cor` argument.

```
gst.camera <- camera(v, index=c2.ind, design=design, contrast = cont.matrix[,1], inter.gene.cor=0.05)
```

CAMERA outputs a dataframe of the resulting statistics, with each row denoting a different gene set. The output is ordered by p-value so that the most significant should be at the top. Let's look at the top 5 gene sets:

```
gst.camera[1:5,]
```


	NGenes	
REACTOME_PEPTIDE_CHAIN_ELONGATION	81	
KEGG_RIBOSOME	83	
REACTOME_3_UTR_MEDIATED_TRANSLATIONAL_REGULATION	102	
REACTOME_FORMATION_OF_THE_TERNARY_COMPLEX_AND_SUBSEQUENTLY_THE_43S_COMPLEX	48	
REACTOME_SRP_DEPENDENT_COTRANSLATIONAL_PROTEIN_TARGETING_TO_MEMBRANE	108	
	Direction	
REACTOME_PEPTIDE_CHAIN_ELONGATION	Up	
KEGG_RIBOSOME	Up	
REACTOME_3_UTR_MEDIATED_TRANSLATIONAL_REGULATION	Up	
REACTOME_FORMATION_OF_THE_TERNARY_COMPLEX_AND_SUBSEQUENTLY_THE_43S_COMPLEX	Up	
REACTOME_SRP_DEPENDENT_COTRANSLATIONAL_PROTEIN_TARGETING_TO_MEMBRANE	Up	
	PValue	
REACTOME_PEPTIDE_CHAIN_ELONGATION	3.711614e-10	
KEGG_RIBOSOME	4.242401e-10	
REACTOME_3_UTR_MEDIATED_TRANSLATIONAL_REGULATION	1.484636e-09	
REACTOME_FORMATION_OF_THE_TERNARY_COMPLEX_AND_SUBSEQUENTLY_THE_43S_COMPLEX	1.477539e-08	
REACTOME_SRP_DEPENDENT_COTRANSLATIONAL_PROTEIN_TARGETING_TO_MEMBRANE	1.716413e-08	
	FDR	
REACTOME_PEPTIDE_CHAIN_ELONGATION	1.001631e-06	
KEGG_RIBOSOME	1.001631e-06	
REACTOME_3_UTR_MEDIATED_TRANSLATIONAL_REGULATION	2.336816e-06	
REACTOME_FORMATION_OF_THE_TERNARY_COMPLEX_AND_SUBSEQUENTLY_THE_43S_COMPLEX	1.620981e-05	
REACTOME_SRP_DEPENDENT_COTRANSLATIONAL_PROTEIN_TARGETING_TO_MEMBRANE	1.620981e-05	

The total number of significant gene sets at 5% FDR is

```
table(gst.camera$FDR < 0.05)
```

```
FALSE TRUE
4593  129
```

You can write out the camera results to a csv file to open in excel.

```
write.csv(gst.camera, file="gst_BPregVsLac.csv")
```

Challenge

1. Run `camera` on the second contrast in the contrast matrix.
2. Run `camera` on a different set of MSigDB gene sets, the hallmark datasets, `mouse_H_v5.rdata`. You will need to load in the hallmark gene sets, and the object will be called `Mm.H` in R.

Solution

```
FALSE TRUE
45      5
```

	NGenes	Direction	PValue
HALLMARK_MYC_TARGETS_V2	60		Up 7.859018e-09
HALLMARK_MYC_TARGETS_V1	204		Up 3.706364e-07
HALLMARK_E2F_TARGETS	201		Up 1.198872e-04
HALLMARK_G2M_CHECKPOINT	199		Up 6.547050e-04
HALLMARK_OXIDATIVE_PHOSPHORYLATION	199		Up 2.493738e-03
HALLMARK_TGF_BETA_SIGNALING	53		Up 1.068929e-02
HALLMARK_UNFOLDED_PROTEIN_RESPONSE	110		Up 1.669518e-02
HALLMARK_CHOLESTEROL_HOMEOSTASIS	73	Down	1.956095e-02
HALLMARK_ALLOGRAFT_REJECTION	159		Up 5.451666e-02
HALLMARK_MTORC1_SIGNALING	201		Up 1.088344e-01
		FDR	
HALLMARK_MYC_TARGETS_V2	3.929509e-07		
HALLMARK_MYC_TARGETS_V1	9.265909e-06		
HALLMARK_E2F_TARGETS	1.998120e-03		
HALLMARK_G2M_CHECKPOINT	8.183812e-03		
HALLMARK_OXIDATIVE_PHOSPHORYLATION	2.493738e-02		
HALLMARK_TGF_BETA_SIGNALING	8.907745e-02		
HALLMARK_UNFOLDED_PROTEIN_RESPONSE	1.192513e-01		
HALLMARK_CHOLESTEROL_HOMEOSTASIS	1.222560e-01		
HALLMARK_ALLOGRAFT_REJECTION	3.028704e-01		
HALLMARK_MTORC1_SIGNALING	5.441718e-01		

ROAST gene set testing

ROAST is an example of a self-contained gene set test (D. Wu et al. 2010). It asks the question, “Do the genes in my set tend to be differentially expressed between my conditions of interest?”. ROAST doesn't care about what the other genes in the experiment are doing, which is different to `camera` and `goana`. ROAST is a good option for when you're interested in a specific set, or a few sets. It is not really used to test thousands of sets at one time.

From the Hallmark gene sets, two MYC pathways were most significant.

```
H.camera[1:10,]
```

	NGenes	Direction	PValue
HALLMARK_MYC_TARGETS_V2	60		Up 7.859018e-09
HALLMARK_MYC_TARGETS_V1	204		Up 3.706364e-07
HALLMARK_E2F_TARGETS	201		Up 1.198872e-04
HALLMARK_G2M_CHECKPOINT	199		Up 6.547050e-04
HALLMARK_OXIDATIVE_PHOSPHORYLATION	199		Up 2.493738e-03
HALLMARK_TGF_BETA_SIGNALING	53		Up 1.068929e-02
HALLMARK_UNFOLDED_PROTEIN_RESPONSE	110		Up 1.669518e-02
HALLMARK_CHOLESTEROL_HOMEOSTASIS	73	Down	1.956095e-02
HALLMARK_ALLOGRAFT_REJECTION	159		Up 5.451666e-02
HALLMARK_MTORC1_SIGNALING	201		Up 1.088344e-01
		FDR	
HALLMARK_MYC_TARGETS_V2	3.929509e-07		
HALLMARK_MYC_TARGETS_V1	9.265909e-06		
HALLMARK_E2F_TARGETS	1.998120e-03		
HALLMARK_G2M_CHECKPOINT	8.183812e-03		
HALLMARK_OXIDATIVE_PHOSPHORYLATION	2.493738e-02		
HALLMARK_TGF_BETA_SIGNALING	8.907745e-02		
HALLMARK_UNFOLDED_PROTEIN_RESPONSE	1.192513e-01		
HALLMARK_CHOLESTEROL_HOMEOSTASIS	1.222560e-01		
HALLMARK_ALLOGRAFT_REJECTION	3.028704e-01		
HALLMARK_MTORC1_SIGNALING	5.441718e-01		

Let's see if there are any MYC signalling pathways in MsigDB C2 collection. We can do this with the `grep` command on the names of the gene sets.

```
grep("MYC_",names(c2.ind))
```

```
[1] 496 544 621 1591 1592 1783 1784 2029 2030 2031 2032 2194 2195 2203
[15] 2204 2435 2436 2506 2507 2600 2609 2620 2621 2629 2676 2694 2706 2716
[29] 2727 2744 2786 2824 3311 3401 3817 3943 3944 4204 4205 4206 4207 4433
[43] 4434 4616
```

```
# Let's save these so that we can subset c2.ind to test all gene sets with MYC in the name
myc <- grep("MYC_",names(c2.ind))
# What are these pathways called?
names(c2.ind)[myc]
```

```
[1] "PID_MYC_ACTIV_PATHWAY"
[2] "PID_MYC_PATHWAY"
[3] "PID_MYC_REPRESS_PATHWAY"
[4] "ODONNELL_TARGETS_OF_MYC_AND_TFRC_UP"
[5] "ODONNELL_TARGETS_OF_MYC_AND_TFRC_DN"
[6] "CAIRO_PML_TARGETS_BOUND_BY_MYC_UP"
[7] "CAIRO_PML_TARGETS_BOUND_BY_MYC_DN"
[8] "SCHLOSSER_MYC_AND_SERUM_RESPONSE_SYNERGY"
[9] "SCHLOSSER_MYC_TARGETS_REPRESSED_BY_SERUM"
[10] "SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_DN"
[11] "SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_UP"
[12] "CEBALLOS_TARGETS_OF_TP53_AND_MYC_UP"
[13] "CEBALLOS_TARGETS_OF_TP53_AND_MYC_DN"
[14] "KIM_MYC_AMPLIFICATION_TARGETS_UP"
[15] "KIM_MYC_AMPLIFICATION_TARGETS_DN"
[16] "BENPORATH_MYC_TARGETS_WITH_EBOX"
[17] "BENPORATH_MYC_MAX_TARGETS"
[18] "MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_UP"
[19] "MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN"
[20] "SCHUHMACHER_MYC_TARGETS_DN"
[21] "LEE_LIVER_CANCER_MYC_TGFA_DN"
[22] "LEE_LIVER_CANCER_MYC_UP"
[23] "MENSSEN_MYC_TARGETS"
[24] "LEE_LIVER_CANCER_MYC_TGFA_UP"
[25] "LEE_LIVER_CANCER_MYC_E2F1_DN"
[26] "LEE_LIVER_CANCER_MYC_E2F1_UP"
[27] "SCHUHMACHER_MYC_TARGETS_UP"
[28] "LEE_LIVER_CANCER_MYC_DN"
[29] "COLLER_MYC_TARGETS_UP"
[30] "COLLER_MYC_TARGETS_DN"
[31] "YU_MYC_TARGETS_UP"
[32] "YU_MYC_TARGETS_DN"
[33] "BILD_MYC_ONCOGENIC_SIGNATURE"
[34] "SANSOM_APC_MYC_TARGETS"
[35] "SOUCEK_MYC_TARGETS"
[36] "ELLWOOD_MYC_TARGETS_UP"
[37] "ELLWOOD_MYC_TARGETS_DN"
[38] "DANG_REGULATED_BY_MYC_UP"
[39] "DANG_REGULATED_BY_MYC_DN"
[40] "DANG_MYC_TARGETS_UP"
[41] "DANG_MYC_TARGETS_DN"
[42] "ACOSTA_PROLIFERATION_INDEPENDENT_MYC_TARGETS_UP"
[43] "ACOSTA_PROLIFERATION_INDEPENDENT_MYC_TARGETS_DN"
[44] "ALFANO_MYC_TARGETS"
```

Let's use ROAST to see if these MYC related gene sets tend to be differentially expressed. Note that the syntax for `camera` and `roast` is almost identical.

```
myc.rst <- roast(v,index=c2.ind[myc],design=design,contrast=cont.matrix[,1],nrot=999)
myc.rst[1:15,]
```

	NGenes	PropDown	PropUp
MENSSEN_MYC_TARGETS		54	0.11111111 0.7592593
CAIRO_PML_TARGETS_BOUND_BY_MYC_UP		24	0.08333333 0.7500000
DANG_REGULATED_BY_MYC_UP		69	0.10144928 0.7101449
DANG_MYC_TARGETS_UP		144	0.09722222 0.7013889
SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_UP		48	0.12500000 0.6458333
PID_MYC_ACTIV_PATHWAY		79	0.12658228 0.6455696
SCHLOSSER_MYC_TARGETS_REPRESSED_BY_SERUM		156	0.12820513 0.6217949
LEE_LIVER_CANCER_MYC_UP		50	0.30000000 0.5400000
KIM_MYC_AMPLIFICATION_TARGETS_UP		176	0.14772727 0.5284091
BILD_MYC_ONCOGENIC_SIGNATURE		187	0.27272727 0.4973262
BENPORATH_MYC_TARGETS_WITH_EBOX		216	0.21759259 0.4259259
MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN		16	0.37500000 0.3125000
COLLER_MYC_TARGETS_UP		24	0.04166667 0.8333333
SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_DN		48	0.06250000 0.7916667
YU_MYC_TARGETS_UP		43	0.06976744 0.7674419
		Direction	PValue FDR
MENSSEN_MYC_TARGETS		Up	0.001 0.001833333
CAIRO_PML_TARGETS_BOUND_BY_MYC_UP		Up	0.001 0.001833333
DANG_REGULATED_BY_MYC_UP		Up	0.001 0.001833333
DANG_MYC_TARGETS_UP		Up	0.001 0.001833333
SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_UP		Up	0.001 0.001833333
PID_MYC_ACTIV_PATHWAY		Up	0.001 0.001833333
SCHLOSSER_MYC_TARGETS_REPRESSED_BY_SERUM		Up	0.001 0.001833333
LEE_LIVER_CANCER_MYC_UP		Up	0.001 0.001833333
KIM_MYC_AMPLIFICATION_TARGETS_UP		Up	0.001 0.001833333
BILD_MYC_ONCOGENIC_SIGNATURE		Up	0.001 0.001833333
BENPORATH_MYC_TARGETS_WITH_EBOX		Up	0.001 0.001833333
MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN		Down	0.001 0.001833333
COLLER_MYC_TARGETS_UP		Up	0.002 0.004125000
SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_DN		Up	0.002 0.004125000
YU_MYC_TARGETS_UP		Up	0.002 0.004125000
		PValue.Mixed	FDR.Mixed
MENSSEN_MYC_TARGETS		0.001	0.001
CAIRO_PML_TARGETS_BOUND_BY_MYC_UP		0.001	0.001
DANG_REGULATED_BY_MYC_UP		0.001	0.001
DANG_MYC_TARGETS_UP		0.001	0.001
SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_UP		0.001	0.001
PID_MYC_ACTIV_PATHWAY		0.001	0.001
SCHLOSSER_MYC_TARGETS_REPRESSED_BY_SERUM		0.001	0.001
LEE_LIVER_CANCER_MYC_UP		0.001	0.001
KIM_MYC_AMPLIFICATION_TARGETS_UP		0.001	0.001
BILD_MYC_ONCOGENIC_SIGNATURE		0.001	0.001
BENPORATH_MYC_TARGETS_WITH_EBOX		0.001	0.001
MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN		0.001	0.001
COLLER_MYC_TARGETS_UP		0.002	0.002
SCHLOSSER_MYC_TARGETS_AND_SERUM_RESPONSE_DN		0.002	0.002
YU MYC TARGETS UP		0.001	0.001

Each row corresponds to a single gene set. The NGenes column gives the number of genes in each set. The PropDown and PropUp columns contain the proportions of genes in the set that are down- and up-regulated, respectively, with absolute fold changes greater than 2. The net direction of change is determined from the significance of changes in each direction, and is shown in the Direction column. The PValue provides evidence for whether the majority of genes in the set are DE in the specified direction, whereas the PValue.Mixed tests for differential expression in any direction. FDRs are computed from the corresponding p-values across all sets.

Challenge

1. Test whether the MYC signalling pathways tend to be differentially expressed between luminal pregnant vs lactating (the second contrast).
2. Look for gene sets containing “WNT” in the name and see whether they tend to be differentially expressed in basal pregnant vs lactating.

Solution

	NGenes	PropDown	PropUp
ELLWOOD_MYC_TARGETS_DN		40	0.5750000 0.25000000
DANG_MYC_TARGETS_DN		30	0.1666667 0.56666667
MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN		16	0.5625000 0.31250000
PID_MYC_PATHWAY		25	0.2400000 0.48000000
LEE_LIVER_CANCER_MYC_E2F1_DN		42	0.4761905 0.23809524
PID_MYC_REPRESS_PATHWAY		59	0.2542373 0.45762712
KIM_MYC_AMPLIFICATION_TARGETS_DN		82	0.3048780 0.45121951
BENPORATH_MYC_TARGETS_WITH_EBOX		216	0.3194444 0.43518519
LEE_LIVER_CANCER_MYC_TGFA_DN		44	0.4318182 0.27272727
CEBALLOS_TARGETS_OF_TP53_AND_MYC_UP		22	0.3636364 0.09090909
ODONNELL_TARGETS_OF_MYC_AND_TFRC_DN		45	0.6666667 0.11111111
CEBALLOS_TARGETS_OF_TP53_AND_MYC_DN		33	0.3030303 0.45454545
BILD_MYC_ONCOGENIC_SIGNATURE		187	0.2994652 0.45989305
YU_MYC_TARGETS_UP		43	0.7209302 0.11627907
ALFANO_MYC_TARGETS		220	0.3090909 0.42272727
	Direction	PValue	FDR
ELLWOOD_MYC_TARGETS_DN	Down	0.001	0.002444444
DANG_MYC_TARGETS_DN	Up	0.001	0.002444444
MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN	Down	0.001	0.002444444
PID_MYC_PATHWAY	Up	0.001	0.002444444
LEE_LIVER_CANCER_MYC_E2F1_DN	Down	0.001	0.002444444
PID_MYC_REPRESS_PATHWAY	Up	0.001	0.002444444
KIM_MYC_AMPLIFICATION_TARGETS_DN	Up	0.001	0.002444444
BENPORATH_MYC_TARGETS_WITH_EBOX	Up	0.001	0.002444444
LEE_LIVER_CANCER_MYC_TGFA_DN	Down	0.001	0.002444444
CEBALLOS_TARGETS_OF_TP53_AND_MYC_UP	Down	0.002	0.006600000
ODONNELL_TARGETS_OF_MYC_AND_TFRC_DN	Down	0.003	0.009166667
CEBALLOS_TARGETS_OF_TP53_AND_MYC_DN	Up	0.003	0.009166667
BILD_MYC_ONCOGENIC_SIGNATURE	Up	0.004	0.011846154
YU_MYC_TARGETS_UP	Down	0.005	0.014142857
ALFANO_MYC_TARGETS	Up	0.006	0.016133333
	PValue.Mixed	FDR.Mixed	
ELLWOOD_MYC_TARGETS_DN	0.001	0.001	
DANG_MYC_TARGETS_DN	0.001	0.001	
MORI_EMU_MYC_LYMPHOMA_BY_ONSET_TIME_DN	0.001	0.001	
PID_MYC_PATHWAY	0.001	0.001	
LEE_LIVER_CANCER_MYC_E2F1_DN	0.001	0.001	
PID_MYC_REPRESS_PATHWAY	0.001	0.001	
KIM_MYC_AMPLIFICATION_TARGETS_DN	0.001	0.001	
BENPORATH_MYC_TARGETS_WITH_EBOX	0.001	0.001	
LEE_LIVER_CANCER_MYC_TGFA_DN	0.001	0.001	
CEBALLOS_TARGETS_OF_TP53_AND_MYC_UP	0.001	0.001	
ODONNELL_TARGETS_OF_MYC_AND_TFRC_DN	0.001	0.001	
CEBALLOS_TARGETS_OF_TP53_AND_MYC_DN	0.001	0.001	
BILD_MYC_ONCOGENIC_SIGNATURE	0.001	0.001	
YU_MYC_TARGETS_UP	0.002	0.002	
ALFANO_MYC_TARGETS	0.001	0.001	

	NGenes	PropDown	PropUp	Direction	
LABBE_WNT3A_TARGETS_UP		104	0.2307692	0.49038462	Up
PID_WNT_NONCANONICAL_PATHWAY		31	0.4516129	0.09677419	Down
WILLERT_WNT_SIGNALING		22	0.2272727	0.54545455	Up
SANSOM_WNT_PATHWAY_REQUIRE_MYC		53	0.1886792	0.45283019	Up
PID_WNT_CANONICAL_PATHWAY		19	0.4736842	0.21052632	Down
REACTOME_SIGNALING_BY_WNT		62	0.1290323	0.50000000	Up
BIOCARTA_WNT_PATHWAY		24	0.2500000	0.33333333	Up
ST_WNT_BETA_CATENIN_PATHWAY		32	0.3125000	0.34375000	Up
KEGG_WNT_SIGNALING_PATHWAY		131	0.3740458	0.25190840	Down
LABBE_TARGETS_OF_TGFB1_AND_WNT3A_UP		100	0.3400000	0.38000000	Up
PID_WNT_SIGNALING_PATHWAY		23	0.3043478	0.30434783	Up
LABBE_TARGETS_OF_TGFB1_AND_WNT3A_DN		90	0.2555556	0.40000000	Up
ST_WNT_CA2_CYCLIC_GMP_PATHWAY		14	0.2857143	0.35714286	Up
LABBE_WNT3A_TARGETS_DN		65	0.2615385	0.27692308	Up
WNT_SIGNALING		75	0.3600000	0.30666667	Down
		PValue	FDR	PValue.Mixed	
LABBE_WNT3A_TARGETS_UP		0.001	0.0037500	0.001	
PID_WNT_NONCANONICAL_PATHWAY		0.001	0.0037500	0.001	
WILLERT_WNT_SIGNALING		0.002	0.0075000	0.001	
SANSOM_WNT_PATHWAY_REQUIRE_MYC		0.004	0.0131250	0.001	
PID_WNT_CANONICAL_PATHWAY		0.014	0.0405000	0.001	
REACTOME_SIGNALING_BY_WNT		0.018	0.0437500	0.005	
BIOCARTA_WNT_PATHWAY		0.044	0.0875000	0.001	
ST_WNT_BETA_CATENIN_PATHWAY		0.049	0.0875000	0.001	
KEGG_WNT_SIGNALING_PATHWAY		0.053	0.0875000	0.001	
LABBE_TARGETS_OF_TGFB1_AND_WNT3A_UP		0.066	0.0982500	0.001	
PID_WNT_SIGNALING_PATHWAY		0.086	0.1165909	0.001	
LABBE_TARGETS_OF_TGFB1_AND_WNT3A_DN		0.170	0.2118750	0.001	
ST_WNT_CA2_CYCLIC_GMP_PATHWAY		0.248	0.2855769	0.001	
LABBE_WNT3A_TARGETS_DN		0.487	0.5212500	0.001	
WNT_SIGNALING		0.890	0.8900000	0.001	
		FDR.Mixed			
LABBE_WNT3A_TARGETS_UP		0.001			
PID_WNT_NONCANONICAL_PATHWAY		0.001			
WILLERT_WNT_SIGNALING		0.001			
SANSOM_WNT_PATHWAY_REQUIRE_MYC		0.001			
PID_WNT_CANONICAL_PATHWAY		0.001			
REACTOME_SIGNALING_BY_WNT		0.005			
BIOCARTA_WNT_PATHWAY		0.001			
ST_WNT_BETA_CATENIN_PATHWAY		0.001			
KEGG_WNT_SIGNALING_PATHWAY		0.001			
LABBE_TARGETS_OF_TGFB1_AND_WNT3A_UP		0.001			
PID_WNT_SIGNALING_PATHWAY		0.001			
LABBE_TARGETS_OF_TGFB1_AND_WNT3A_DN		0.001			
ST_WNT_CA2_CYCLIC_GMP_PATHWAY		0.001			
LABBE_WNT3A_TARGETS_DN		0.001			
WNT_SIGNALING		0.001			

Notes

- A common application of ROAST is to use a set of DE genes that was defined from an analysis of an independent data set. ROAST can then determine whether similar changes are observed in the contrast of interest for the current data set.
- Even for GO-defined gene sets, goana and ROAST have different behaviours. In goana, the significance of differential expression for a GO term is determined relative to other DE genes that are not annotated with that term. In ROAST, only differential expression for the genes in the set are relevant to the significance of that set and its corresponding term. goana depends on a significance cutoff to choose DE genes, whereas ROAST does not require a cutoff and evaluates all genes in the set.
- ROAST estimates p-values by simulation, so the results may change slightly between runs. More precise p-values can be obtained by increasing the number of rotations, albeit at the cost of increased computational time.
- The smallest p-value that can be reported is $1/(2nrot + 1)$ where nrot is the number of rotations. This lower bound can be decreased by increasing nrot.

Visualising gene set tests: Barcode and enrichment plots

A barcode plot can be produced with the `barcodeplot` function to visualize the results for any particular set. To display a barcodeplot, we need to decide what statistics to use, usually we choose either logFC or t-statistics for the comparison of interest. We also need to make sure we give the statistics in the correct order, such that subsetting the statistics vector will give the genes for the gene set we're interested in. The best way to do this is to work with the fit object directly. The `coefficients` slot contains the logFCs and the `t` slot contains the t-statistics.

Let's have a look at one of the top MYC related pathway in the ROAST test that you have already done, "MENSSEN_MYC_TARGETS".

```
# Have a look at the logFCs and t-statistics in fit.cont
names(fit.cont)
```

```
[1] "coefficients"      "stdev.unscaled"    "sigma"
     [4] "df.residual"      "cov.coefficients"  "rank"
     [7] "Amean"            "method"            "design"
    [10] "contrasts"        "df.prior"          "s2.prior"
    [13] "var.prior"        "proportion"        "s2.post"
    [16] "t"                "df.total"          "p.value"
    [19] "lods"             "F"                 "F.p.value"
    [22] "genes"
```

```
head(fit.cont$coefficients)
```

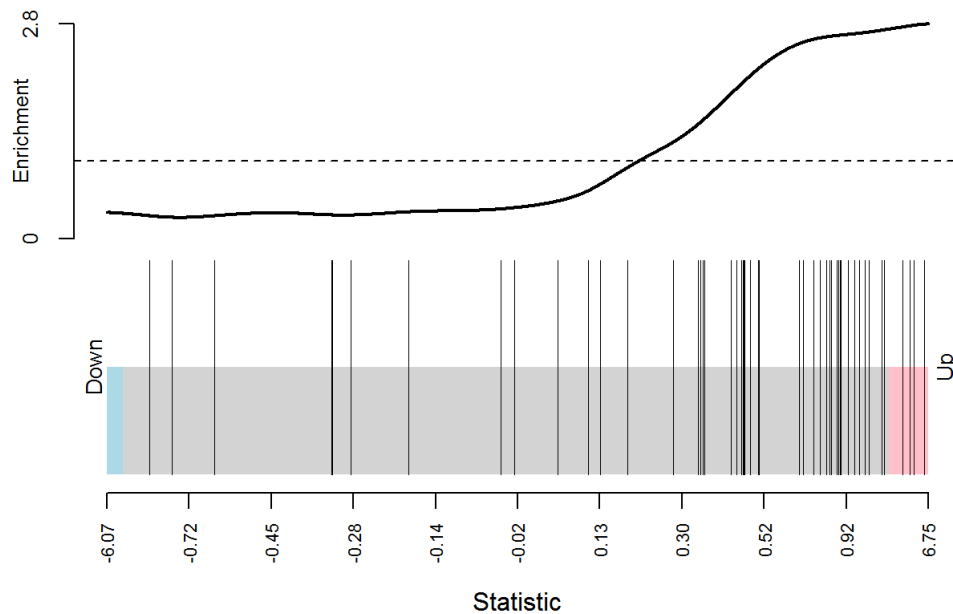
```
Contrasts
      B.PregVsLac L.PregVsLac
497097 -1.62084708 -0.6964826
20671  0.41298109  0.3652171
27395 -0.03540567 -0.3849389
18777 -0.08505647 -0.4731888
21399  0.21912852  0.2839044
58175  1.49852621  1.4736067
```

```
head(fit.cont$t)
```

```
Contrasts
      B.PregVsLac L.PregVsLac
497097 -4.0541969 -0.3076765
20671  1.0030050  0.4055901
27395 -0.1936036 -2.2395329
18777 -0.7975228 -4.0746018
21399  2.1866392  2.5284968
58175  1.5788218  3.1387631
```

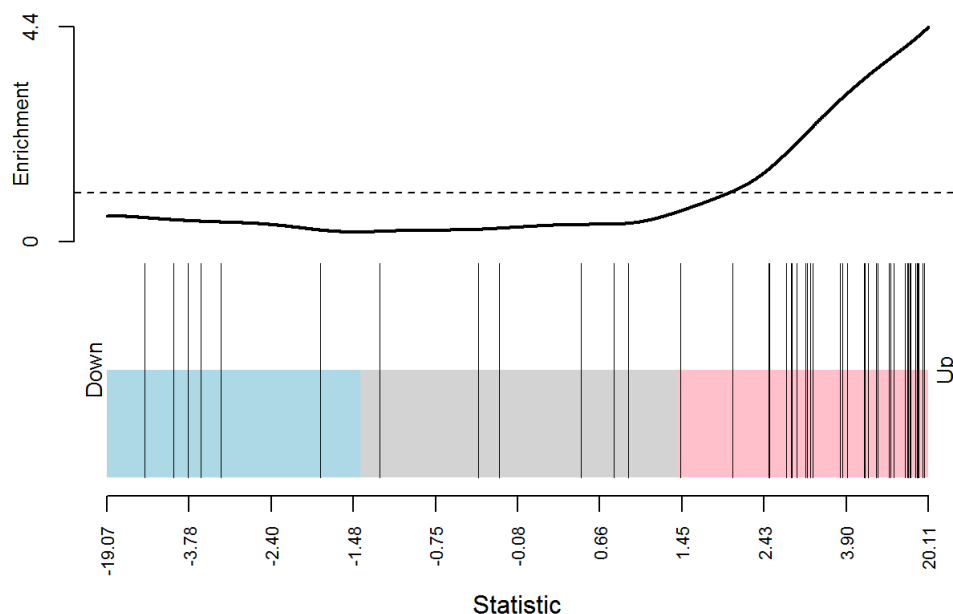
```
par(mfrow=c(1,1))
# barcode plot with logFCs
barcodeplot(fit.cont$coeff[,1], index=c2.ind[["MENSSEN_MYC_TARGETS"]], main="LogFC: MENSSEN_MYC_TARGETS")
```

LogFC: MENSSEN_MYC_TARGETS



```
# barcode plot using t-statistics
barcodeplot(fit.cont$t[,1], index=c2.ind[["MENSSEN_MYC_TARGETS"]], main="T-statistic: MENSSEN_MYC_TARGETS")
```

T-statistic: MENSSEN_MYC_TARGETS



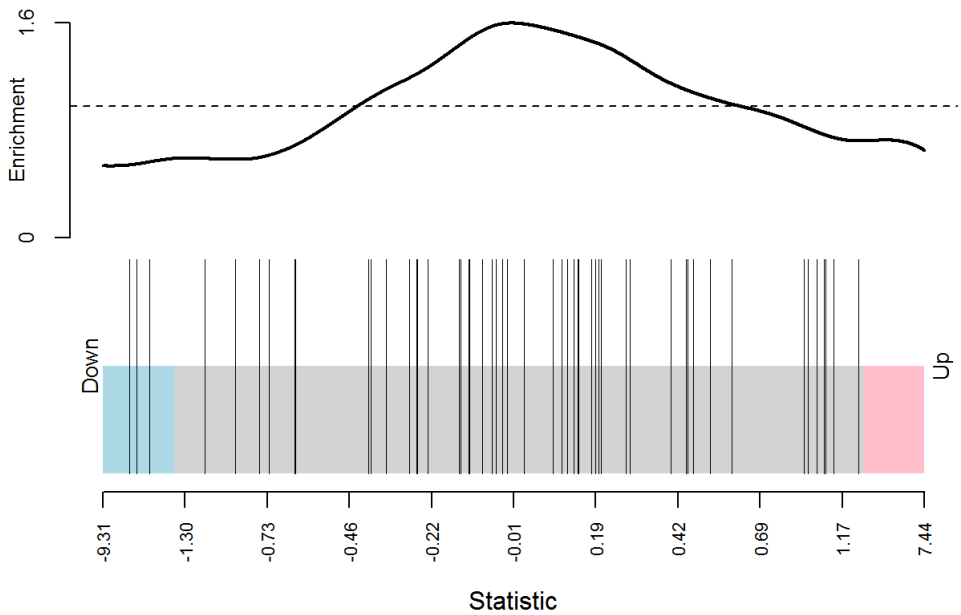
Here, genes are represented by bars and are ranked from left to right by decreasing log-fold change or t-statistic. This forms the barcode-like pattern. The line above the barcode shows the relative local enrichment of the vertical bars in each part of the plot. The barcodeplot shows that the genes in this gene set tend to be up-regulated between pregnant and lactating in basal cells.

Challenge

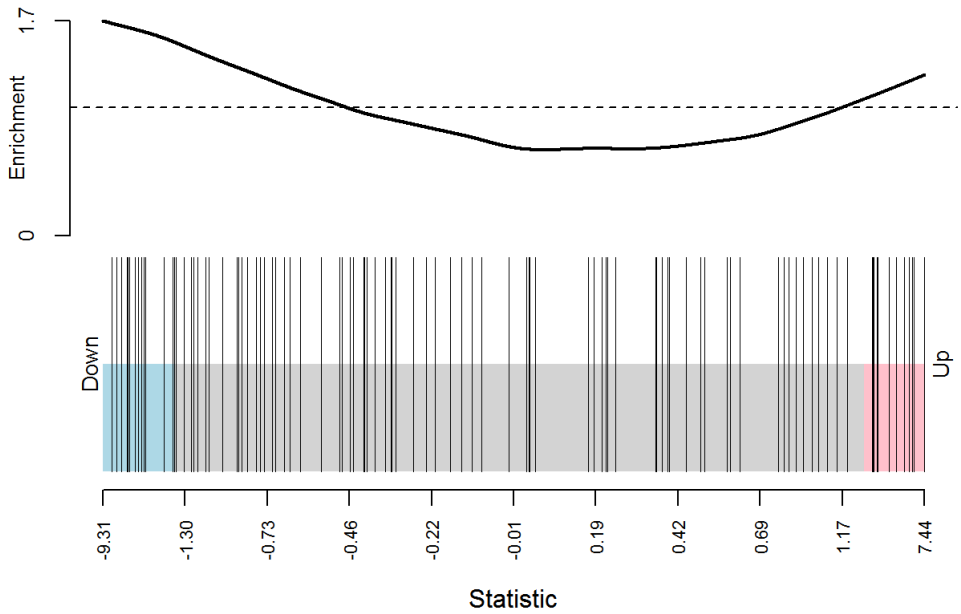
1. Produce a barcodeplot for luminal pregnant vs lactating for this gene set. Does the pattern of enrichment look as strong?
2. Choose one of the Wnt signalling pathways and produce a barcode plot for both comparisons.
3. You can put two gene sets on one plot, for example a set that is up-regulated and one that is down-regulated, by adding a gene set to the `index2` argument. Produce a barcodeplot with two sets of your choosing.

Solution

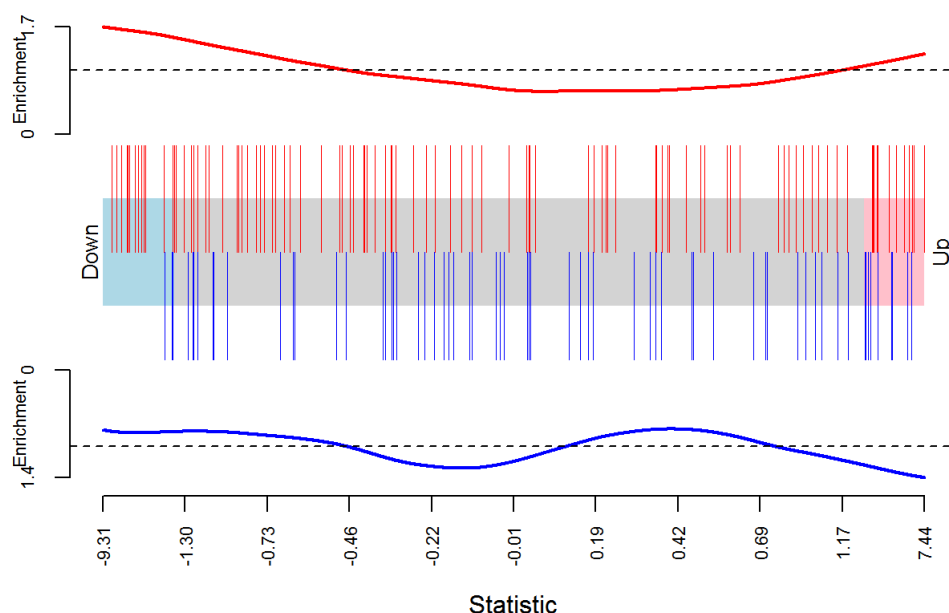
L.PregVsLac: MENSSEN_MYC_TARGETS



L.PregVsLac: LABBE_WNT3A_TARGETS_UP



L.PregVsLac: LABBE_WNT3A_TARGETS UP and DN sets



Record package and version info with sessionInfo()

The session information describes the versions of R and of the packages that were used in the analysis. This is useful for record-keeping purposes, and ensures that an analysis can be reproduced even when the software is updated over time.

```
sessionInfo()
```

```
R version 3.4.0 (2017-04-21)
Platform: x86_64-w64-mingw32/x64 (64-bit)
Running under: Windows 7 x64 (build 7601) Service Pack 1

Matrix products: default

locale:
[1] LC_COLLATE=English_Australia.1252 LC_CTYPE=English_Australia.1252
[3] LC_MONETARY=English_Australia.1252 LC_NUMERIC=C
[5] LC_TIME=English_Australia.1252

attached base packages:
[1] parallel stats4      stats      graphics  grDevices  utils      datasets
[8] methods   base

other attached packages:
[1] RColorBrewer_1.1-2  org.Mm.eg.db_3.4.1  AnnotationDbi_1.37.4
[4] IRanges_2.9.19      S4Vectors_0.13.17   Biobase_2.35.1
[7] BiocGenerics_0.21.3 gplots_3.0.1        Glimma_1.3.22
[10] edgeR_3.17.10       limma_3.31.22       knitr_1.16

loaded via a namespace (and not attached):
[1] Rcpp_0.12.10      magrittr_1.5        lattice_0.20-35
[4] BiasedUrn_1.07    stringr_1.2.0       caTools_1.17.1
[7] tools_3.4.0       grid_3.4.0          KernSmooth_2.23-15
[10] DBI_0.6-1         htmltools_0.3.5     gtools_3.5.0
[13] yaml_2.1.14       rprojroot_1.2       digest_0.6.12
[16] bitops_1.0-6      memoise_1.1.0       RSQLite_1.1-2
[19] evaluate_0.10     rmarkdown_1.5       gdata_2.17.0
[22] stringi_1.1.5     GO.db_3.4.1         compiler_3.4.0
[25] backports_1.1.0   locfit_1.5-9.1      jsonlite_1.4
```

References

- Fu, Nai Yang, Anne C Rios, Bhupinder Pal, Rina Soetanto, Aaron T L Lun, Kevin Liu, Tamara Beck, et al. 2015. "EGF-mediated induction of Mcl-1 at the switch to lactation is essential for alveolar cell survival." *Nature Cell Biology* 17 (4): 365–75. doi:10.1038/ncb3117 (<https://doi.org/10.1038/ncb3117>).
- Law, Charity W, Yunshun Chen, Wei Shi, and Gordon K Smyth. 2014. "Voom: precision weights unlock linear model analysis tools for RNA-seq read counts." *Genome Biology* 15 (2): R29. doi:10.1186/gb-2014-15-2-r29 (<https://doi.org/10.1186/gb-2014-15-2-r29>).
- Liao, Yang, Gordon K Smyth, and Wei Shi. 2014. "featureCounts: an efficient general purpose program for assigning sequence reads to genomic features." *Bioinformatics (Oxford, England)* 30 (7): 923–30. doi:10.1093/bioinformatics/btt656 (<https://doi.org/10.1093/bioinformatics/btt656>).
- Lun, Aaron T L, Yunshun Chen, and Gordon K Smyth. 2016. "It's DE-licious: A Recipe for Differential Expression Analyses of RNA-seq Experiments Using Quasi-Likelihood Methods in edgeR." *Methods in Molecular Biology (Clifton, N.J.)* 1418 (January): 391–416. doi:10.1007/978-1-4939-3578-9_19 (https://doi.org/10.1007/978-1-4939-3578-9_19).
- McCarthy, Davis J, and Gordon K Smyth. 2009. "Testing significance relative to a fold-change threshold is a TREAT." *Bioinformatics (Oxford, England)* 25 (6): 765–71. doi:10.1093/bioinformatics/btp053 (<https://doi.org/10.1093/bioinformatics/btp053>).
- Ritchie, M. E., B. Phipson, D. Wu, Y. Hu, C. W. Law, W. Shi, and G. K. Smyth. 2015. "limma powers differential expression analyses for RNA-sequencing and microarray studies." *Nucleic Acids Research*, January, gkv007. doi:10.1093/nar/gkv007 (<https://doi.org/10.1093/nar/gkv007>).
- Robinson, M D, D J McCarthy, and G K Smyth. 2010. "edgeR: a Bioconductor package for differential expression analysis of digital gene expression data." *Bioinformatics* 26 (1). Oxford Univ Press: 139–40.
- Robinson, Mark D, and Alicia Oshlack. 2010. "A scaling normalization method for differential expression analysis of RNA-seq data." *Genome Biology* 11 (3): R25. doi:10.1186/gb-2010-11-3-r25 (<https://doi.org/10.1186/gb-2010-11-3-r25>).
- Wu, D, and G K Smyth. 2012. "Camera: a competitive gene set test accounting for inter-gene correlation." *Nucleic Acids Research* 40 (17). Oxford University Press: e133—e133.
- Wu, D, E Lim, F Vaillant, M L Asselin-Labat, J E Visvader, and G K Smyth. 2010. "ROAST: rotation gene set tests for complex microarray experiments." *Bioinformatics* 26 (17). Oxford Univ Press: 2176–82.